Omni-Compass

The Live Engine Manual

How to read, operate, tune and watch the Omni-Compass live engine, native beside Omni-Compass, section by section, monitor by monitor and knob by knob

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` *and* `DISCLOSURES.md`. `SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0` *All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Page 1

Table of contents

Table of contents 2

Front matter 6

What this book is . . . 6

How to use the book with the page . . . 6

Part I. Reading the live engine 7

1. The two arms: native, and Omni-Compass on top of native . . . 7

2. The key: GAIN, EVEN, LOSS and power given . . . 7

3. Evidence classes and the state of a monitor . . . 9

4. The kill switch and the master switch . . . 10

5. The control panel and the knobs . . . 10

6. The monitor wall . . . 12

7. The Python's runs, in the page . . . 12

8. Grafana, Prometheus and the open-source feeds . . . 13

9. The organisms and the whole stack, every way it is bought . . . 13

Part II. The live engine, section by section 14

iii Title page and build . . . 14

iv Executive summary and reading key . . . 14

Part 1 of the page: The monitor wall and the control panel . . . 15

1a The monitor wall and the control panel: the kill switch, every monitor, the status board . . . 15

1b CI pipeline: GitHub Actions runs in progress and queued . . . 21

2 The benefit board: every domain, native against Omni-Compass . . . 22

3 The whole organism, every way it is bought: on-site bare metal, through the cloud, in the device, as a service . . . 24

4 Measured effect in full: the organisms, every muscle, the register, the grid and every three-run table 26

Part 5 of the page: Evidence by stack, from the cloud down to the chip . . . 29

5a Cloud providers: Azure, Google Cloud, Amazon Web Services . . . 29

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

5b Orchestrators and governors: every scheduler, queue and multi-cluster governor, native and with Omni-Compass on top . . . 33
5c Kubernetes: official run playback . . . 34
5d Kubernetes: per-repetition run telemetry . . . 38
5e Kubernetes and cloud: the shipped controller's decision trace . . . 40
5f Hardware: the GPU card and the CPU host . . . 42
Part 6 of the page: The organism, live: native and Omni-Compass . . . 45
6a The organism, live: native and Omni-Compass . . . 45
6b Telemetry analytics and Omni-Compass in its own words . . . 48
7 Control console: every runtime control of the live engine . . . 53
Part 8 of the page: Control architecture: the brain . . . 54
8a One control cycle, traced step by step . . . 54
8b Phase plane and scoreboard . . . 55
8c The brain's internal state . . . 58
8d The verdict: change acceptance testing . . . 61
9 Failure modes, controls and residual risk . . . 63
10 Component specification and mathematics . . . 64
11 Self-governed rendering: Omni-Compass inside this page . . . 65
v Disclosures, governing law and contact . . . 66

Part III. Operating it as the engineer in charge **67**

Watching: the first minute and the first hour . . . 67
When a row reads LOSS . . . 67
Tuning and tweaking . . . 67
What waits, and what fills it . . . 68
Rebuilding the page and the book . . . 68

Back matter **69**

Appendix A. Every monitor . . . 69
Appendix B. Every knob . . . 72

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Appendix C. Every LOSS row, whose it is, its cause and the knob that answers it 74

Appendix D. Glossary 86

Contact 87

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Front matter

What this book is

The third of the three manuals of Omni-Compass. The Operator's Manual shows an engineering team how to embed, wire, tune, run and check Omni-Compass on a stack; the Owner's Manual holds the whole mechanism, its mathematics, its source and every result. This book reads the live engine, `docs/engine/OMNI_COMPASS_LIVE_ENGINE.html`, from its first line to its last, so that an operator can sit at the page with the book open beside it and run it as the engineer in charge: read every gauge, turn every knob, press the switch, watch the organism, and know what every monitor shows, where its numbers come from, how strongly they stand, and what is still waiting for its run.

The page and this book were built from the same commit: the engine that stands is omni-v12 (digest d38b58fbe28939b8), the page built 2026-10-20 19:19 UTC from commit ad993defc2a8. The book is written from the page itself (`docs/book/build_live_manual.py`): every section, number, title, summary and control line below is the page's own, every monitor is the page's own canvas under its own number, so the two read each other both ways. When the page changes, the book is rebuilt with it.

How to use the book with the page

- Open the page in a browser (the repository's copy, or its published copy). It runs entirely in the browser: it reads no live system, sends nothing anywhere, and draws every view from the records embedded in it at build time and from the engine's own mathematics running in it.
- The page opens with its table of contents, under the one line of the legal notice; then the front matter (the title page and the key to every reading); then eleven sections from the whole organism down to the smallest part; then the back matter. Part II of this book follows the same order and the same numbers.
- Every monitor (every chart canvas) carries its number in a badge above it, from 1 at the top of the page to 57 at the bottom. The monitor wall in section 1a shows them all at once, numbered, read right to left and then down; this book lists each under its number with what it shows, its evidence, the open-source feed that would carry it on a running system, and its knobs.
- Every section opens with a summary and a controls line saying whether its views are live (the engine running in the page, moved by the console's knobs) or recorded (a run or a table played back, which no knob can change).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Part I. Reading the live engine

1. The two arms: native, and Omni-Compass on top of native

Every view on the page has two sides and only two. **Native** is the system running by itself under its own controllers: the Kubernetes autoscaler, the card's own driver, the kernel's frequency governor, the cloud's own autoscaler, each muscle's own controller. **Omni-Compass** is the same system with Omni-Compass sitting on top of those native controllers, never in place of them. Native is drawn in amber, Omni-Compass in blue. Where a monitor draws two lanes, native is the left lane and Omni-Compass the right, on one scale, so that a line lying on the other never hides it. A red dashed line is the gauge's own limit (the response line, a temperature line). A red tick is a write the governor made.

Omni-Compass is a supervisory governor: its one goal is to strengthen the organism and keep it healthy, every muscle working as far as its own limit and never past it (do no harm, `DISCLOSURES.md` section 3). The engine that stands keeps the strictest form of that rule: every muscle is judged on itself, never driven below native, nothing traded between muscles; where it cannot bring a knob a GAIN, the knob stays native.

2. The key: GAIN, EVEN, LOSS and power given

Every judged reading in Omni-Compass is one of three words: GAIN, EVEN or LOSS. Better, same and worse are not used anywhere. The word is decided by what the gauge is for, never by the plus or minus sign of its number: who sets the aim differs from one system to the next, and each gauge's aim is written in its preregistration before the runs.

GAIN: G Growth (asset appreciation); A Advantage (risk reduction); I Improvement (margin expansion); N Net-positive (profitable outcome)

- underwriting: surplus and positive variance
- in a table: the gauge moved toward its declared aim against native: more work done, a shorter wait, fewer misses or failures, or a resource cut

EVEN: E Equalized (zero variance); V Verified baseline (expected norm); E Equilibrium (balanced risk); N Neutral (no net change)

- underwriting: the break-even baseline: equilibrium
- in a table: native and Omni could not be told apart: the same, or no difference beyond the noise; a knob native already runs at the top of its range is EVEN, at its limit on native

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

LOSS: L Leakage (capital drain); O Off-track (adverse selection); S Setback (claim liability); S Subpar (negative variance)

- underwriting: actuarial deficit and exposure
- in a table: the gauge moved away from its declared aim: less work done, a longer wait, more misses or failures

How the sign of a number becomes a word:

- Time waited, missed or failed (a response time, time over the line, failed requests): a fall is GAIN, a rise is LOSS.
- Work done (work inside the line, throughput, a hit rate): a rise is GAIN, a fall is LOSS.
- A resource spent or held (energy, the power drawn and its peaks and swings, machines, memory, processor time, a bill, wear on a part): a fall is a cut, and a cut is GAIN. A rise is power given: what Omni put in to buy the work and the speed shown beside it. Power given is always shown and never hidden, and it is never called a loss (the founder's order of 12 October 2026: the max rides every muscle's edge; it never shaves). A peak is power held, so a peak that rose is power given, in a data centre and in a building alike (17 October).
- Work bought with each unit of a resource (work per energy): a rise is a cut in what each unit of work took, a GAIN; a fall is power given, because it falls when more power is given for the same work.
- So a minus can read GAIN (a response time cut 65%, a bill cut 30%) and a plus can read LOSS (failed requests up).
- The floor (the founder's order of 20 October 2026): a change under half of one percent (0.5%) either way, in every run, reads EVEN, under the floor; the row keeps its measured size. From 0.5% up, a change that holds the same way in all three runs reads GAIN or LOSS by the gauge's aim.
- The sign beside a word (the same order): the number next to a word carries the word's sign, never the gauge's raw direction, so that no one mistakes which way a number went: GAIN +, LOSS −, EVEN =. A bill cut 3% reads GAIN +3%, failed requests up 2% read LOSS −2%, and a change inside the noise reads EVEN =0.3%.

Some preregistrations name their outcome with a label written before the runs (for example SUPERIOR WITHIN GUARDRAILS in the card protocol). The label stays as it was frozen, and every table shows it with its word first: TOWARD ITS AIMS reads GAIN; BOTH WAYS reads GAIN AND LOSS; AWAY FROM ITS AIMS reads LOSS; SAME AS NATIVE reads EVEN (the same as native); AT ITS LIMIT ON NATIVE reads EVEN (at its limit on native); SUPERIOR WITHIN GUARDRAILS reads GAIN (within the guardrails); ENERGY IMPROVEMENT WITH SERVICE TRADEOFF reads GAIN AND LOSS (energy cut, service traded for it); SERVICE IMPROVEMENT WITH ENERGY TRADEOFF reads GAIN, power given (service GAIN, energy given for it); NONINFERIOR / INCONCLUSIVE reads

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

EVEN (inside the noise, within the guardrails); NOT ESTABLISHED reads EVEN (not established: inside the noise, a guardrail crossed on the mean). A run that breaks its own validity checks reads INVALID and carries no word.

A reading counts only when it holds the same way in all three of our runs, A, B and C, with every 95% interval clear of zero: "GAIN in all three of our runs". The other readings are EVEN, no difference beyond the noise (in a run the interval includes zero, so native and Omni could not be told apart: that is the result), the runs disagree (clear runs pointing different ways: the test is unstable there and is looked into) and shown, not judged (a gauge declared in advance as shown only, such as the knob's own setting).

Why these three words: they are the words of underwriting and actuarial work, where every outcome is graded as surplus, break-even or deficit, and each letter spells out the same rule. Omni-Compass is there to add (GAIN), never to take (LOSS), and to hand a muscle back to its own native setting where it cannot add (EVEN). Its first rule is do no harm, in the founder's meaning of 19 October 2026: the organism Omni-Compass governs is never left turning out less than its native controllers make it turn out, nor less healthy, and every muscle works only as far as its own limit (DISCLOSURES.md, section 3). So LOSS should never appear: where it does, the row stays in the table, its cause is found and fixed (our own wiring first, then native's setup, then the scoring) and the test is run again.

Disclosure and disclaimer: GAIN, EVEN and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results. The underwriting senses explain the words; they are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. A reading states what our own runs measured against native under the declared aims, not a promise about any other system; every number stands beside its word, so a reader can judge it by any other convention. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results. Preregistered means the rules were written down in our own repository before the runs; nothing was filed with any outside body.

The founder's floor of 20 October: a change under 0.5% either way in every run reads EVEN, under the floor. The number beside a word carries the word's sign (GAIN +, LOSS −, EVEN =), never the gauge's raw direction. Colours follow the word everywhere on the page: GAIN green, EVEN yellow, LOSS red, power given blue.

3. Evidence classes and the state of a monitor

Every view says its evidence class: **P**, a physical meter (a card's own power meter, a processor's energy counter); **L**, live software (real Kubernetes, real databases, a real cloud, run on GitHub's own machines or in the cloud, played back from their own SHA-256-checked files); **S**, the repository's own models (the engine's harness, the toy cluster, the organisms, the grid), never presented as proof. Every monitor also

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

says its state:

- **live**: the engine's own mathematics running in the page (the brain, the compass law, the verdict), proven equal to the Python and the C++ to 1e-12 on every build; the console's knobs move it.
- **replay**: a recorded run played back from its own files, second by second or decision by decision; no knob changes it.
- **record** or **composed**: the repository's tables read at build time, or composed from them by a stated law.
- **model**: a model of a machine standing in for a run not yet made (the CPU host), marked as such on the monitor itself.
- **awaiting its run**: built before its run; its native lane shows what exists, its Omni-Compass lane stays empty, and the page names the run that fills it. Nothing on such a monitor is invented.

4. The kill switch and the master switch

One switch governs the whole organism, at the top, above the brain (`omnicompass/master.py`, `tools/omni_switch.py`). While Omni-Compass governs, the switch reads **ON** in green. Pressed, the switch turns **red**: at the next decision every governor puts every setting it wrote back, reads each one back and stops; the word **OFF** stands in white, and every Omni-Compass side of every monitor follows native, drawn in native's colour (a lighter tint of amber, dashed, so both lines stay visible). Native's own controllers under every muscle never stop and never see the switch. A recorded result never changes with it: its colour does, never a value, a word or a row.

The same switch stands in the control panel at the head of section 1a, at the head of every section, in every monitor's own knobs and in the console (section 7): one press anywhere is one press everywhere. It is for security, and it is instant. The walk back after a crash is a different mechanism, the lease, which returns every setting to native gradually. The master switch's own panel in section 1a draws the wiring and lists every governor on the page with what it handed back.

5. The control panel and the knobs

The control panel at the top of section 1a holds the kill switch, the button that runs the Python's own runs in the page, the three manuals and the wall as a Grafana dashboard. The console (section 7) holds every runtime knob of the live engine; every monitor that is live carries the same knobs in its own strip under its canvas, so the organism can be driven from wherever one is looking. Every knob acts on both arms alike: the difference between the lanes is Omni-Compass's doing alone.

- **Kill switch** (kill): Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); on the master switch (section 1b).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **The dial** (dial): the max (every muscle to its own limit) or the tune-down (never more than native); on the console's dial.
- **Demand** (load): ×0.25 to ×2 of the day's own demand, both arms alike; on the console's demand slider.
- **Noise** (noise): σ of the demand's noise, one draw a muscle a decision, both arms alike; on the console's noise slider.
- **Sense delay** (lag): decisions added between what a muscle does and what its controller reads, both arms alike; on the console's delay slider.
- **Faults** (faults): a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; on the console's fault buttons.
- **Speed** (speed): 1×, 4×, 16× or 60× of the engine's clock; on the console's time knob.
- **One decision** (step): step the engine one decision while paused; on the console's time knob.
- **Pause** (pause): hold the engine's clock; on the console's time knob.
- **Frame load** (frame): +0, +8 or +16 ms burned inside every frame of this page; on the console's frame-load knob.
- **Run** (run): A, B or C (or the archived run) the monitor plays; on the monitor's own buttons.
- **Repetition** (rep): which repetition of the run is replayed; on the monitor's own buttons.
- **Arm** (arm): which arm stands on the right: Omni-Compass governing, or watching only; on the monitor's own buttons.
- **Stack** (stack): the GPU card, the Linux host, Kubernetes or the cloud; on the monitor's own buttons.
- **Governor** (governor): which orchestrator or governor is shown; on the monitor's own table.
- **Scenario** (scenario): on-site bare metal, through the cloud, in the device, as a service; on the monitor's own buttons.
- **Reading** (reading): every measured piece carried, or the index's strict rule; on the monitor's own buttons.
- **The chip's run** (chip): the chip's one run beside the count or in it; on the monitor's own buttons.
- **Gauge** (gauge): which gauge of the run is shown; on the monitor's own buttons.
- **Filter** (filter): which rows, words, families or organisms are shown; on the monitor's own buttons.
- **Body** (body): which body the organism runs: the harness, the organism, every muscle alone; on the organism's own buttons.
- **Muscle** (muscle): which muscle's own card is read; on the organism's own table.
- **Runs** (runs): how many of the Python's runs this page computes again; on the monitor's own buttons.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Print** (print): the receipts of the runs, printable, with the digests and the notice; on the monitor's own button.
- **Reading direction** (direction): the wall read right to left (as set) or left to right; on the wall's own button.
- **Seed base** (seed): which held-out seed base's 500 scenarios are replayed; on the monitor's own buttons.

6. The monitor wall

The wall (section 1a) shows every monitor of the page at once: 57 canvases, each a tile carrying its number and its section. They are numbered in their order of appearance from the top of the page and laid out right to left, then the next row (the founder's order; a button reads them left to right instead). A tile with an accent border is a monitor on screen now. The page's own governor (section 11) rests a monitor while it is off screen; while the wall is in view the resting monitors are drawn in turn, one a frame. A click on a tile goes to the monitor. The inventory under the wall lists every monitor with its feed and its knobs, and names every monitor still awaiting its run.

7. The Python's runs, in the page

The proof monitors in section 1a tie the page to the Python. `tools/engine_runs.py` runs N runs from a base seed: run k's seed is base + k·0x9E3779B9 mod 2^{32}, its random stream mulberry32 (the page's own generator, written in the Python bit for bit). Each run steps the engine that stands through 288 decisions of the brain's state equations integrated by the fourth-order Runge–Kutta step, 80 steps of the compass law and one give-back trial judged by the verdict, and its receipt holds the brain's final state, its pushes and grants, the law's mean and last force and the trial's answer. The digest is the SHA-256 of every receipt at six decimals. The record in this build: — runs from base —, digest —…

Press **Run the Python's 500 runs here** and the page computes the same runs with its own port of the engine, in the browser's idle time, compares every receipt with the Python's and computes its own digest: the two must be one. **A new set** runs any other base in the page and prints the one command that runs the same set in the Python (`python3 tools/engine_runs.py --runs N --base B`), whose digest must equal the page's. **Print the receipts** opens every receipt, the Python's beside the page's, with both digests and the notice, ready for the printer; **Download** saves them as a file.

Beside them the page replays the Python's 500 scenarios a seed of `benchmarks/stack_benchmark.py` (the held-out seeds): each scenario through every arm, native beside the chosen Omni-Compass arm, every scenario's trace hash and the record's own SHA-256. These are replayed from the record, never recomputed in the page; `verify.py` replays them on every push and `verify.py --full-replay` regenerates the whole file byte for byte.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

8. Grafana, Prometheus and the open-source feeds

Every monitor on the page is the kind of panel an operator already runs in Grafana. The registry `docs/engine/monitors.json` names for each the open-source exporter and the Prometheus series that would carry it on a running system: node_exporter for the machines and the processor (its rapl and cpufreq collectors), kube-state-metrics for the cluster's objects, the NVIDIA DCGM exporter for the card, Kepler for energy by pod, OpenCost for the bill, the service's own request histograms for the golden signals, sloth or pyrra for the service level, and the governor's own exporter (`tools/omni_exporter.py`, read-only, from its audit) for every decision, write, hold and switch state. `docs/engine/export_grafana.py` writes the whole wall as one Grafana dashboard (`deploy/grafana/omni-compass-wall.json`): a panel a monitor, in the page's order and numbering, each on its series, beside the governor's own screen (`deploy/grafana/omni-compass.json`) and its alert rules (`deploy/prometheus/omni-compass-rules.yaml`). A monitor the page draws from the repository's tables or from its own clock becomes a text panel saying so. The control panel downloads the dashboard; the page itself reads no live system.

9. The organisms and the whole stack, every way it is bought

The catalog's seven organisms (the spine, 267 muscles, the base every stack runs on; the four realms; the four stacked; the whole tower of 1,107 muscles) each run native and with Omni-Compass on top, under their own load, the climb and work waiting in line. The orchestra stands beside them, every governor's muscles as one body. Section 3 composes the whole stack, the software, the cloud and the hardware, into one body: the founder counts it as the eighth organism running native and the ninth with Omni-Compass on top of every piece. It is composed four ways: on-site bare metal (software × bare metal), through the cloud (cloud × bare metal × software, and the cloud as two pieces), in the device (the bare metal alone) and as a service (the software alone).

Through the cloud, the bare metal at the back end belongs to the provider: no tenant can meter it, so no run reaches it. The cloud piece is measured as what the provider meters and bills (the machines and the bill, native against Omni-Compass on top of the cloud's own autoscaler), which is also the provider's own benefit on its own hardware. The bare-metal piece is gauged on hardware we can meter and carried to the provider's bare metal only as a modelled factor of the same mechanism on the same kind of part, never as a measurement of the provider's machines (`DISCLOSURES.md` section 9). No run has stacked the layers yet: the composition is class S of class P and L measurements until the stacked run, preregistered in `docs/WHOLE_SYSTEM_PREREGISTRATION.md`, replaces it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Part II. The live engine, section by section

Each chapter is one section of the page, in the page's order and under its number: what the page says it is, its controls, its panels, and every monitor in it under its own number, with what it shows, its evidence and state, the feed that would carry it live, its knobs, how to read it and what to do with it.

iii Title page and build

The page, in its own words.

A live monitor of the Omni-Compass engine and of every benchmark GitHub has run on it: each run played back from its own SHA-256-checked files, each model run by the repository's own code, so the engine can be watched moving instead of only read. Two sides everywhere: Native, the whole system running by itself, and Omni-Compass, the same system with Omni-Compass on top of all of it. It is laid out as a book, read from the top down: the table of contents first, then the front matter (this title page and the reading key), then the monitor wall with the control panel and the kill switch, every monitor at once with its number and its knobs, then the benefit board, then the whole organism composed every way it is bought, then every part from the whole down to the smallest: the measured effect in full, the evidence by stack from the cloud down to the chip, the organism and its muscles, the control console, the brain and its verdict, the failure modes, the component specification, and this page's own rendering. The disclosures close it. The engine that stands is omni-v12: omni-v11 with two faults found on its own runs fixed. A processor-limit raise that the cluster's admission refuses is held until the pod's limit changes, the raise shrinks or a wait passes; omni-v11 wrote it again at every fast check. The swarm's proximity reflex hands back the planner's cruise in the tick it is triggered. omni-v11's guard stays in it: a reading that shows no machine in service is blind, and nothing is decided or written from it.

DISCLOSURE: REAL RUNS, A REAL ENGINE, NOTHING HARDWIRED. Every real-run figure on this page is played back from the raw files of benchmark runs executed on GitHub's own machines, each file checked against its run's SHA-256 list before a single number is drawn. Every model view runs the engine's own code (the compass law, the brain, the verdict) live in this page, linked through software to the repository's Python and C++ and checked against them on every build. GitHub rebuilds the page from the repository whenever the engine, a model or a result changes, and every three hours: no number on it is typed in by hand, and when the repository changes, the page changes with it.

Its monitors. This section draws no canvas: its views are tables and text the page builds from its records.

iv Executive summary and reading key

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Controls. Live: the buttons act on the live engine; every number below is read from the records on this page.

The page, in its own words.

Executive summary

Every number in this list is read from the records on this page when it opens (the three-run tables, the Omni index, the cloud ledger, the card run, the models' own readings, the realms table and the harness); none is typed in. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

The key: GAIN, EVEN and LOSS

The terms this page uses, each defined once (the first time a term appears on the page it carries the same definition, shown when pointed at)

The size is how far the gauge moved from native; the word says whether that move helped the work. Examples, read from the three-run tables on this page, appear here when they load. A miss is a decision where the work went over its line. The key is written once in tools/readings.py and stands at the top of the README, the reading guide and both manuals. GAIN, EVEN and LOSS, and the letters behind them, are The Omni-Compass LLC's own convention for reading its own results; the underwriting senses explain the words and are not an actuarial, financial or insurance opinion, and no outside body defined, reviewed or approved them. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

Quick-start reading guide

Its monitors. This section draws no canvas: its views are tables and text the page builds from its records.

Part 1 of the page: The monitor wall and the control panel

1a The monitor wall and the control panel: the kill switch, every monitor, the status board

The first view after the front matter: every monitor of this page at once, numbered, and the control panel that drives them. The kill switch stands at its head; the Python's own runs can be run again here at a button; the status board below says whether Omni-Compass is engaged, which signals are out of tolerance and where, and every LOSS row is raised as an alarm with its source record, its cause and the knob that answers it. No value on the board is typed in: each is computed at runtime from the live engine or from the repository's records embedded at build time.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Controls. Live: the kill switch, the faults and the filters act at once, and every tile is read again four times a second.

The page, in its own words.

The control panel: one switch for the whole organism, the Python's runs in this page, the three manuals

The kill switch. While Omni-Compass governs on top of native the switch reads ON in green. Pressed, it turns red: at the next decision every governor puts every setting it wrote back, reads each one back and stops (omnicompass/master.py, tools/omni_switch.py); the word OFF stands white, and every Omni-Compass side of every monitor follows native, drawn in native's colour. Native's own controllers under every muscle never stop and never see the switch. A recorded result never changes with it: its colour does, never a value, a word or a row. The same switch stands at the head of every section and in every monitor's own knobs; one press anywhere is one press everywhere. It is for security and is instant; the walk back after a crash is the lease, separate.

The three manuals. The Live Engine Manual reads this page top to bottom, monitor by monitor and knob by knob: how to read, operate, tune and watch it, native beside Omni-Compass. The Operator's Manual wires and runs Omni-Compass on a stack; the Owner's Manual holds the whole mechanism, its mathematics, its source and every result. Each carries the notice, the LICENSE, the NOTICE and DISCLOSURES.md.

The monitor wall: every monitor of this page, numbered in its order of appearance, read right to left; click a tile to go to it

The same canvases the sections draw, copied small four times a second; nothing is drawn twice from different data. The page's own governor (section 11) leaves a monitor resting while it is off screen, so while this wall is in view the resting monitors are drawn in turn, one a frame, and a tile with the accent border is a monitor on screen now.

Every monitor: the inventory: what each shows, where its numbers come from, the open-source feed that would carry it live, its knobs, and what is still awaiting its run

Every monitor is one canvas on this page; its number is its order of appearance from the top, and the wall reads them right to left, then the next row. The feeds name the open-source exporters an operator's Prometheus and Grafana already run (node_exporter, kube-state-metrics, the NVIDIA DCGM exporter, Kepler, OpenCost, the governor's own tools/omni_exporter.py); the button in the control panel downloads the whole wall as a Grafana dashboard on those series (deploy/grafana/omni-compass-wall.json, written from docs/engine/monitors.json), beside the governor's own screen (deploy/grafana/omni-compass.json). A monitor the page draws from the repository's tables, or from its own clock, says so: no series carries it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The master switch, wired at the top of the organism: switch, brain, organism, every muscle

One switch for the whole organism, as the repository builds it (omnicompass/master.py, tools/omni_switch.py). It sits at the top and holds the brain's outputs. With it OFF, nothing the brain computes is pushed or pulled into any muscle, and every muscle runs on native's own controllers, which never stop. On a real machine every governor exits at the switch; on this page the brain keeps reading the native body in shadow, so its monitors keep moving, and nothing it computes leaves it. No monitor is switched off: every Omni-Compass side follows native, drawn in native's colour. Every governor on this page is registered with it and obeys it at once. The live organism hands every setting it held back to native's own controller and reads each one back. Every record's Omni-Compass side, and every playback, trace, table, board and grid, reads native's own. Native's own controllers sit under every muscle and are never switched, so native's side never moves with it. Every section carries the same switch in its controls bar; pressing any one of them throws this one.

The Python's runs, in this page: the engine's seeded runs recomputed here, and the 500 scenarios replayed

How the two prove each other. tools/engine_runs.py runs N runs from a base seed: run k's seed is base + k·0x9E3779B9 mod 2^{32} (the jump generator), its stream mulberry32 (this page's own generator, written in the Python bit for bit), through the engine that stands: 288 decisions of the brain's state equations integrated by the fourth-order Runge–Kutta step, 80 steps of the compass law and one give-back trial judged by the verdict. Each run's receipt is its final state, its pushes and grants, the law's mean and last force and the trial's answer; the digest is the SHA-256 of every receipt at six decimals. This page runs the same N with its own port of the engine (proven equal to the Python and the C++ to 1e-12 on every build by docs/engine/check_live_engine.py) and compares every receipt and the digest. Run the Python again with another base and it writes another set; this page computes the same set from the same base, and the two digests must be one.

Alarms and watch items, by severity

Every LOSS row: its cause and the knob that answers it (Omni-Compass's own rows first, then the orchestrators' own cost)

Golden-signal feed, live: the last 240 decisions, native beside Omni-Compass

One point per decision streamed from the live engine, gapless, each signal on its own axis, with each arm's last, mean and maximum as a time-series dashboard reports them; a kill-switch window is shaded red. Native in amber, Omni-Compass in blue, the same demand on both.

Engine event log, newest first

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The live engine's own events as it emits them, with the wall-clock time and the decision. They are a threshold breach and its clearing on either arm, every injected fault and its end, the kill switch, the dial, a new day, and the brain's calm crossing the grant threshold. ALERT, WARN and INFO as an operations log grades them.

OpenMetrics exposition: the live metrics, ready for Prometheus and Grafana

The live engine's metrics in the OpenMetrics text format, the CNCF exposition format Prometheus scrapes and Grafana charts, refreshed every two seconds. They are engagement, the decision index, breaches, work, band time, actuator moves and reversals per arm, and frame time. Then the verdict's keep ratio at each true cost, the three-run tables' rows by class and reading, and the build's engine, digest and commit.

The board is laid out the way an operations monitor is. The switch and the engine come first. Next come the live organism's signals, native beside Omni-Compass, in the four a service is watched by (site reliability practice: latency, traffic, errors and saturation): here the work waiting, the work done, the decisions past their line and the capacity held. Then every reading of the real and modelled runs that went against native, worst first. Last come GitHub and the age of this page. A tile's word is its reading against native by the key (front matter, iv). A tile with no reading says what it shows.

Its monitors.

Monitor 1: The master switch and the kill switch

The switch above the brain: ON, Omni-Compass governs every wired muscle; engaged, every setting is handed back to native's own controller at once and every Omni-Compass side on the page follows native.

- **Evidence and state.** Class live; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_master_switch`; on a running system: `omni_master_switch`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The wiring from the top down: the master switch above the brain, the brain above the organism, the organism above every muscle, and native's own controllers under every muscle, never switched. The switch's state is the word on it: ON in green while Omni-Compass governs; engaged, the switch red and OFF in white.
- **What to do with it.** Press the kill switch only for security: it is instant, every setting goes back to native and is read back. Watch the governors' table beside it: every governor on the page says OFF and what it handed back.

Monitor 2: The Python's runs, reproduced here

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The engine's seeded runs: the Python's receipts (results/engine_runs) beside the same runs computed by this page's own engine, run by run, with one digest each.

- **Evidence and state.** Class S (the engine itself); live.
- **Feed.** tools/engine_runs.py (the Python) and this page.
- **Knobs.** Runs: how many of the Python's runs this page computes again; Print: the receipts of the runs, printable, with the digests and the notice.
- **How to read it.** The Python's seeded runs beside this page's: one cell a run (green agrees to 1e-9, red differs, blue a new set), the two digests under them; and the Python's 500 scenarios a seed replayed with every trace hash.
- **What to do with it.** Press the button to run the Python's set here; run a new base to make a new set and run the printed command in the Python to compare digests. Print or download the receipts for the file.

Monitor 3: The Python's 500 scenarios, replayed

benchmarks/stack_benchmark.py's 500 scenarios a seed base (the two held-out seeds), each through every arm, replayed from its record scenario by scenario with every trace hash and the file's SHA-256; verify.py replays them on every push.

- **Evidence and state.** Class S; replay.
- **Feed.** benchmarks/stack_benchmark.py (results/heldout_seed_*/RUNS.csv).
- **Knobs.** Seed base: which held-out seed base's 500 scenarios are replayed; Arm: which arm stands on the right: Omni-Compass governing, or watching only; Gauge: which gauge of the run is shown; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The Python's seeded runs beside this page's: one cell a run (green agrees to 1e-9, red differs, blue a new set), the two digests under them; and the Python's 500 scenarios a seed replayed with every trace hash.
- **What to do with it.** Press the button to run the Python's set here; run a new base to make a new set and run the printed command in the Python to compare digests. Print or download the receipts for the file.

Monitor 4: Golden signal: errors

Errors a decision on the live organism, native beside Omni-Compass (site reliability's first of four).

- **Evidence and state.** Class S; live.
- **Feed.** the service's own request counter (Prometheus client), kube-state-metrics for pod restarts; series `http_requests_total{code=~"5.."}`, `kube_pod_container_status_restarts_total`; on a running system:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

`sum(rate(http_requests_total{code=~"5.."}[5m]))`.

- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live engine: native in amber, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

Monitor 5: Golden signal: throughput

Work done a decision (traffic), native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** the service's own request counter (Prometheus client); series `http_requests_total`; on a running system: `sum(rate(http_requests_total[5m]))`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live engine: native in amber, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

Monitor 6: Golden signal: latency

Response time a decision, native beside Omni-Compass.

- **Evidence and state.** Class S; live.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Feed.** the service's own duration histogram; series `http_request_duration_seconds_bucket`; on a running system: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live engine: native in amber, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

Monitor 7: Golden signal: saturation

Capacity held against capacity used, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** node_exporter; series `node_cpu_seconds_total`; on a running system: `1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m]))`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The four golden signals of a service, one point a decision of the live engine: native in amber, Omni-Compass in blue, the same demand on both. A red band is a kill-switch window.
- **What to do with it.** Watch latency and errors first: a rise there is the only thing that can read LOSS. Saturation and throughput are the work and the capacity held: capacity is power given when it rises.

1b CI pipeline: GitHub Actions runs in progress and queued

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Every GitHub Actions workflow run in progress or queued at build time. The hardware and cloud runs, with their terminal status, are in sections 5a and 5f; every run on the engine that stands that terminated without a measurement is raised on the status board (section 1a).

Controls. Recorded data: GitHub's runs as read when the page was built.

The page, in its own words.

GitHub Actions, as a monitor: the runs in flight, the Azure set's repetitions, the clouds' gates, the chips

Read at the build from GitHub's own API: a workflow's running runs in solid blue, its waiting ones pale; the Azure set to the paid cap with its repetitions measured so far (the table is written when the set ends); each cloud's hourly gate with the reason its own job wrote. Live, the same lives in the open-source github-actions-exporter for Prometheus.

Workflow runs in progress and queued

The page is rebuilt by GitHub itself (.github/workflows/live-engine.yml, docs/engine/build_live_engine.py) whenever the engine, a model or a result changes, and every three hours. Every record is made again by the repository's own code on the engine that stands, checked against the Python and the C++, and committed. This table is GitHub's own list at that moment.

Its monitors.

Monitor 8: GitHub Actions: the runs in flight

Every workflow run in progress or queued when the page was built, the Azure set's repetitions done of its cap, and the clouds' gates.

- **Evidence and state.** Class record; record.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter; series `github_workflow_run_status`, `github_job_duration_seconds`; on a running system: `sum by (workflow) (github_workflow_run_status == 1)`.
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** GitHub's runs in flight at the build by workflow, the Azure set's repetitions measured, each cloud's gate with its reason, the chips' runs.
- **What to do with it.** A gate that holds names its fix in its own words; nothing is billed while it holds.

2 The benefit board: every domain, native against Omni-Compass

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The first thing after the wall: every benchmark's measured effect in one board, native against Omni-Compass on top of native, by domain and by gauge family, and the Omni index that weighs them together. Every bar is its table's own word (GAIN green, EVEN yellow, LOSS red, power given blue) beside its size; every LOSS row stands on it and is answered, with its cause and its knob, on the status board (section 1a). The whole organism composed from these same tables follows in section 3; every table, every muscle and the grid in full in section 4.

Controls. Recorded data: every receipt as the repository holds it. The kill switch recolours a recorded result and never rewrites it; console controls never modify it.

The page, in its own words.

The benefit board: every domain, native against Omni-Compass

Where the benefit lives, gauge by gauge

Every judged row of every newest three-run table (the tables below), sorted into the family of gauge it measures. A CTO reads speed, the service level, failures and work. A CFO reads machines, energy and the bill. A plant engineer reads peaks and swings, wear and accuracy, and misses. Each bar counts the rows that read GAIN in all three runs; the size beside it is the median of those rows' own sizes (the middle of runs A, B and C). Power given and LOSS rows are counted beside them.

Aggregate: one figure for the whole board

Its monitors.

Monitor 9: The benefit board

Every domain's three-run table as one bar: GAIN, EVEN, LOSS and power given, native against Omni-Compass.

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Filter: which rows, words, families or organisms are shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, EVEN yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is EVEN whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Monitor 10: Where the benefit lives, gauge by gauge

The same tables by gauge family: work, speed, misses, resources.

- **Evidence and state.** Class L/S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Filter: which rows, words, families or organisms are shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, EVEN yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is EVEN whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

3 The whole organism, every way it is bought: on-site bare metal, through the cloud, in the device, as a service

Omni-Compass is wired into every piece of a stack through its own two-way plug: the software (the cluster: Kubernetes, its add-ons, its governors and orchestrators; and the services on it: the pools, the consumers, the caches), the bare metal under it (the chip's power limit and clock; the machine's kernel and its own meter) and, where the stack is rented, the cloud that bills the machines. A stack is bought one of two ways: on-site bare metal, the software on the owner's own bare metal (software × bare metal, no cloud); through the cloud, the cloud's build, its bare metal at the back end and the software (cloud × bare metal × software), with the cloud as two pieces beside it (cloud × software, the back-end bare metal left to the provider). Every piece's benefit is added into one collective reading, gauge by gauge (work, the 95th percentile response, machines, energy, the bill), with the receipts under it: what native spent, what Omni-Compass spent, and the benefit, in the units each table measured. The repository composes the layers in one place, tools/whole_system.py, which writes results/WHOLE_SYSTEM.md from the tables that stand: each layer measured against its own native with the layer under it as native runs it; a gauge's ratio oriented so that above 1 is a GAIN at every layer alike; the layers composed by the product law, $R(g) = \prod_\ell R_\ell(g)$, $\log R = \Sigma_\ell \log R_\ell$ (for small benefits the product is their sum), every interval carried (the exact Student t per run; A, B and C pooled by inverse variance; the layers' variances added); two readings of the same numbers (every measured piece: everything measured enters with its interval; the strict reading, the index's rule: only what held the same way in all three of our runs, everything else exactly 1); and the separability assumption tested on the governors' four ways. This section recomputes nothing from the

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

tables: it regroups that file's layers into the two ways (docs/engine/export_whole.py), summing the logarithms of the layers a scenario stacks and adding their variances, as the file's rule composes any set of layers; the one number is the mean of the gauge logarithms, its 95% interval under the file's two bounds. The composition is a model (evidence class S) of class P and class L measurements under the separability assumption, and says so; no run has yet stacked every layer at once, and that stacked run is the measurement that would replace it. The chip's one measured run (2 October, on the card controller since replaced) stands beside the count by default, every row of it shown, as the repository's own headline has it; one click puts it in, until its omni-v12 run on Lambda replaces it. The machine (the kernel, bare metal, Kepler) is not yet run and enters as exactly 1, named. GAIN is green, LOSS red, EVEN yellow, power given blue, on every word, cell and tile.

The page, in its own words.

DISCLOSURE: THE CLOUD'S BACK END IS NOT OURS TO METER. Through the cloud, the bare metal at the back end belongs to the provider: no tenant can read its power meter or write its chips' limits, so no run here reaches it. What a tenant can measure is what the cloud meters and bills: the machines it counts every fifteen seconds and the bill at list price (section 5a); that difference, native against Omni-Compass on top of the cloud's own autoscaler, is the cloud piece's benefit, and it is the provider's own health on its own hardware too, fewer machines held for the same work. The bare-metal piece is gauged on hardware we can meter (the card's own meter, the CPU host's energy counter, section 5f) and claimed for the provider's bare metal only as the same mechanism on the same kind of part: the composition multiplies it in as a modelled factor (class S of class P), never as a measurement of the provider's machines. Four ways are composed: on-site bare metal (the owner's own racks and chips: software × bare metal); through the cloud (the cloud's build, its bare metal at the back end and the software; and the cloud as two pieces, the back end left to the provider); in the device (Omni-Compass built into the bare metal alone, the card and the machine, nothing else wired: for an owner who ships hardware); as a service (the software alone, run for an owner with neither bare metal of their own nor a cloud account: the transport of the software only). The eighth and ninth organisms are this whole stack, native and with Omni-Compass on top of every piece.

The stack, native beside Omni-Compass: work flowing down through the pieces

The collective synopsis: the benefits, gauge by gauge, added together

The composition, piece by piece and layer by layer: each factor with its 95% interval, the whole their product

Bills and receipts: what native spent (the cost), what Omni-Compass spent (the expense), and the benefit, in the units each table measured

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Beyond the count: the governors' tables on this page, the models beside, the separability test, and where the missing pieces land

Its monitors.

Monitor 11: The whole organism, composed

The pieces of a stack, native beside Omni-Compass, each piece's composed factors and the whole, their product, in every way a stack is bought.

- **Evidence and state.** Class S (of P and L); composed.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Scenario: on-site bare metal, through the cloud, in the device, as a service; Reading: every measured piece carried, or the index's strict rule; The chip's run: the chip's one run beside the count or in it; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Bars and tiles in the traffic light of the one key: GAIN green to one side, LOSS red to the other, EVEN yellow on zero, power given blue. A figure is a composition or a table's own word, never a single run's number.
- **What to do with it.** Read the word before the size, and the interval before the word: an interval across zero is EVEN whatever its centre. A LOSS on the board is answered on the status board, with its cause and its knob.

4 Measured effect in full: the organisms, every muscle, the register, the grid and every three-run table

Everything the board in section 2 stands on, from the organisms down to the single muscle: the nine organisms under their three loads, all 1,107 muscles of the catalog one cell each, every registered benchmark still to run, the scale-out grid of copies and runs, and every three-run table in the repository with every row as it is written.

Controls. Recorded data: every receipt as the repository holds it. Console controls never modify it.

The page, in its own words.

Every benchmark's benefit in one place, as the repository's benefit sheet holds it (results/BENEFIT_SHEET.json), with the Omni index that weighs them together (results/OMNI_INDEX.md). Then the grid: every organism at 1, 10, 100 and 1,000 copies, each at 1, 10, 100 and 1,000 runs, modelled and with the real cluster inside. The 1,000 tier runs on GitHub's free runners only; paid machines stay at 100. Each bar and square carries its own word; the size beside the word never

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

carries a sign.

The nine organisms: the seven of the catalog under three loads, then the whole stack native and with Omni-Compass

The founder's count of 20 October: the seven organisms of the catalog (the spine first, the base every stack runs on; the four realms; the four stacked; the whole tower), each run native and with Omni-Compass on top of native, under its own load, the climb (1 2 3 2 3 4 …) and work waiting in line; and the whole organism, the software, the cloud and the hardware stacked into one body (section 3), counted twice: the eighth is that stack running native, the ninth the same stack with Omni-Compass on top of every piece. The orchestra, every governor's muscles as one body, stands beside them (section 5b). The monitor below draws the seven from the realms table; the eighth and ninth are section 3's composition until the stacked run, preregistered in docs/WHOLE_SYSTEM_PREREGISTRATION.md, measures them in one run.

Every muscle of the catalog, one cell each, its word against native

Click a cell to read the muscle: its name, family, realm, template, knob and its word under each load.

Third-party software still to be benchmarked: the register's slots, in order

What these slots are. Every muscle in them (a legged robot's gait, locomotion_speed; a wind turbine's blade pitch, pitch_angle_limit; a mobile robot's travel speed, agv_speed and trajectory_speed) already runs, modelled, in the 1,107-muscle catalog and the seven organisms below, native and with OMNI-COMPASS on top. A slot here is the next step for that kind of muscle: the same test on the outside world's own software (MuJoCo's Unitree Go2, NREL's OpenFAST, ROS 2 Nav2), each queued until its three runs land.

The scale-out grid: seven organisms × 1, 10, 100 and 1,000 copies × 1, 10, 100 and 1,000 runs (the 1,000 tier on GitHub's free runners only)

Modelled (class S): the misses

With the real cluster inside (class L inside the model): the 95th percentile

Every three-run table in the repository, native against Omni-Compass, every row as written (each benchmark's newest engine's table; class L real software, class S outside simulators)

From each benchmark's newest engine's table (results/live/V10_*.md today), read row by row by docs/engine/export_scenarios.py; open a row to read every judged row that is not EVEN. Class L: the real software (Kubernetes on kind, the real databases and broker). Class S: the outside simulators run by our own runner (CityLearn, pandapower on SimBench, MuJoCo Menagerie, gym-pybullet-drones). Rows shown and not judged are left out of the counts; the counts take every table of a receipt, its tuning case included where it has one (a receipt's own summary line counts its untouched cases). Run and scored by

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

Every muscle of the catalog and the seven organisms, native against Omni-Compass (the realms table)

The eighth organism, the orchestra (every governor's catalog muscles, each governor's a body of its own and all of them one body), has its own runner and its own table: section 5b, under the governors.

Every family, by stack: each muscle alone, native and then with Omni-Compass

From results/realms_v5/REALMS_V5.json, the realms runner at the max, its engine and commit named above (tools/run_realms_v5.py, catalog realms/catalog_v5.csv), ten paired seeds per muscle and per organism, the lease after 80% of every run. Misses: the plant decisions over their line, summed over the ten seeds. A muscle reads GAIN when one of its aims (more work, faster, fewer misses) moved beyond the noise under some load with none moving the other way; EVEN at its limit on native when its knob already sits at the top of its range. Energy is power given where it rose. Evidence class S, our own models: a model's energy is not a meter's, and a plant written by the same people who wrote the law is not an independent test. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

Its monitors.

Monitor 12: The seven organisms, three loads

Each organism's misses, work and energy against native under its own load, the climb and the line (the realms table).

- **Evidence and state.** Class S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Each organism or grid square read by the one key: green GAIN, yellow EVEN, red LOSS, grey a single run, dashed not yet run.
- **What to do with it.** A dashed square is pending runner capacity, never rented. A LOSS square names its organism, its size and its runs.

Monitor 13: Every muscle of the catalog

The 1,107 muscles as one matrix, each cell its word against native, by family and realm; a click names the muscle.

- **Evidence and state.** Class S; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Every muscle of the catalog one cell, its word against native on its own load: green GAIN, yellow EVEN (pale: at its limit on native), red LOSS.
- **What to do with it.** Click a cell to read the muscle: its family, realm, template, knob and its word under each load.

Monitor 14: The scale-out grid

Seven organisms × copies × runs: every square's word, modelled and with the real cluster inside.

- **Evidence and state.** Class S/L; record.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Filter: which rows, words, families or organisms are shown.
- **How to read it.** Each organism or grid square read by the one key: green GAIN, yellow EVEN, red LOSS, grey a single run, dashed not yet run.
- **What to do with it.** A dashed square is pending runner capacity, never rented. A LOSS square names its organism, its size and its runs.

Part 5 of the page: Evidence by stack, from the cloud down to the chip

5a Cloud providers: Azure, Google Cloud, Amazon Web Services

In a cloud the one metric is the bill: whether the same workload costs less with Omni-Compass on top. Each provider's four test sets and their status.

Controls. Recorded data: GitHub's own cloud runs as read at the build. The controller below plays live; its run and speed follow the console's clock.

The page, in its own words.

In a cloud, Omni-Compass shows one thing: the same work as native for a smaller bill, never a bigger one. Every cloud set runs Omni-Compass at the tune-down on the machines alone: no pod autoscaler moved, no processor conveyed, no power capped, so the pods are exactly native's. It parks only a machine the cloud already made, after its own trial, and wakes it on a breach. The bill and the machines aim lower; work, speed and failures must read EVEN or GAIN. Azure runs first and is the measure of the other two: the same tuning, mechanism and harness.

Every cloud run GitHub has started, set by set

The bill monitors: Azure, Google Cloud and Amazon Web Services, native against Omni-Compass

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

One monitor a provider, the same harness, mechanism and tuning in all three (Azure is the measure of the other two). Each bar pair is a finished set: native's machine-hours (amber) beside Omni-Compass's (blue), the word by the one key (the bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given), the bill at list price under it. A provider with no finished set shows its empty frame and the reason its hourly gate holds, in its own job's words; nothing is drawn for a run that has not run.

The bill on Azure, run by run, each with the engine it ran on

From 19 October: every Azure set runs up to 100 paired repetitions a run, the paid cap, under the preregistered stopping rule (tools/stop_rule.py). After 10, 20, 40 and 70 repetitions the bill's change is read with its 99% interval. The run stops once the bill is read (GAIN, power given, or EVEN within ±2% of native's bill), and never while the speed or the failed requests read a LOSS. Those runs carry "to the cap, stopping rule" in their title and count as their own A, B and C; the 5-repetition runs below stay as their own engines' record.

The shipped controller in the cloud setting, live: native's machines against Omni-Compass's

The controller every cloud set runs (omni_controller/controller.py --give-back-only, the tune-down, the machines alone), decision by decision on the repository's toy cluster (class S). The cloud's own autoscaler runs in both arms. Omni-Compass only parks a machine the cloud already made, after its trial, and wakes it on a breach. The bill is the machine-minutes in service, counted from the record.

Native

Omni-Compass

Its monitors.

Monitor 15: Azure: the bill, run by run

Machine-hours billed and the bill at list price, native against Omni-Compass, every finished Azure set with its 95% interval, and the running set's repetitions done.

- **Evidence and state.** Class L (metered by Azure); record.
- **Feed.** OpenCost; Azure's own billing export; series `node_total_hourly_cost`, `kube_node_info`; on a running system: `sum(node_total_hourly_cost)`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

Monitor 16: Google Cloud: the gate and the bill

The same monitor for Google Cloud: the hourly gate's reading (the key, the login, the quota) and, once a set runs, its bill native against Omni-Compass.

- **Evidence and state.** Class L; awaiting its run: the first set starts within the hour after the gate passes: the key stored whole (docs/GOOGLE_SETUP.md, section 4).
- **Feed.** OpenCost; Google Cloud Billing export; series `node_total_hourly_cost`; on a running system: `sum(node_total_hourly_cost)`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

Monitor 17: Amazon: the gate and the bill

The same monitor for Amazon Web Services: the hourly gate's reading and, once a set runs, its bill native against Omni-Compass.

- **Evidence and state.** Class L; awaiting its run: the first set starts within the hour after the gate passes: the account on the Paid plan (docs/AMAZON_SETUP.md, section 1a).
- **Feed.** OpenCost; AWS Cost and Usage Report; series `node_total_hourly_cost`; on a running system: `sum(node_total_hourly_cost)`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

Monitor 18: The cloud setting, live: native's machines

The shipped controller's cloud mode on the toy cluster: the cloud's own autoscaler alone.

- **Evidence and state.** Class S; live.
- **Feed.** kube-state-metrics; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `kube_node_info`, `omni_nodes_observed`; on a running system: `count(kube_node_info)`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set lands.

Monitor 19: The cloud setting, live: Omni-Compass's machines

The same autoscaler with Omni-Compass parking a machine only after its trial, waking it on a breach.

- **Evidence and state.** Class S; live.
- **Feed.** kube-state-metrics; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `kube_node_info`, `omni_nodes_observed`, `omni_writes_total`; on a running system: `count(kube_node_info)`; `omni_nodes_observed`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The bill the cloud meters: machine-hours and the bill at list price, native's bar beside Omni-Compass's, each finished set with its word; the running set's repetitions measured so far; each provider's hourly gate with the reason its own job wrote.
- **What to do with it.** The bill aims lower: a fall is a GAIN, inside the noise EVEN, a rise power given. A provider whose gate holds runs nothing and bills nothing; its frame stays empty until its first set

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

lands.

5b Orchestrators and governors: every scheduler, queue and multi-cluster governor, native and with Omni-Compass on top

These are the governors and orchestrators that sit between the work and the cluster: batch schedulers and job queues, multi-cluster governors, the default Kubernetes scheduler, GPU and edge orchestrators, and the rightsizing controllers. Each is benchmarked twice. First it runs native, as its owner ships it. Then it runs with Omni-Compass on top of it, on top of native. Each has three runs, and every row is shown. Every place below is laid out now and fills itself from the repository as each one lands. Until then it says what the repository holds, and nothing more.

Controls. Recorded data: each governor's place as the repository holds it, its runs read from GitHub at the build and played here the moment they land; click a governor (a tile, a row of the road or of the catalog) to hold it.

The page, in its own words.

The road to a result, every governor at once: its register row, its preregistration, its rig on GitHub, runs A, B and C, its three-run table

The orchestration stack, live: Omni-Compass on top, the governor native as shipped, Kubernetes and its nodes beneath

Where each governor sits in the stack. Omni-Compass reads and writes on top of it once its wire is in the repository. The governor runs native as shipped beneath it, and Kubernetes places the work on the nodes. The work dots move at the live engine's demand (section 7): a picture of the flow, not a measurement. Each governor's readings replace this picture the moment its three runs land.

The catalog of governors and orchestrators: what each governs, native as shipped, Omni-Compass on top, and what the repository holds now

The four ways: the organism alone, the governor as shipped, Omni-Compass on the organism alone, Omni-Compass on top of the governor, all four in every repetition

The orchestra, the eighth organism: every governor's catalog muscles, native and with Omni-Compass on top (modelled, class S)

Its monitors.

Monitor 20: The orchestrators' road

Every scheduler, queue and multi-cluster governor on its way through trial, A, B and C, on the engine that stands.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Evidence and state.** Class L; record.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter.
- **Knobs.** Governor: which orchestrator or governor is shown.
- **How to read it.** Each scheduler, queue and multi-cluster governor four ways: the organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, Omni-Compass on the orchestrator.
- **What to do with it.** Only two of the four comparisons are Omni-Compass against its native (on top of the orchestrator; on the organism alone); the others are the orchestrator's own cost and are never counted as Omni-Compass's.

Monitor 21: An orchestrator, four ways

The organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, Omni-Compass on the orchestrator: the four arms of the chosen governor.

- **Evidence and state.** Class L; record.
- **Feed.** kube-state-metrics; each orchestrator's own metrics endpoint (Kueue, Volcano, KAI, Koordinator, Crane, KubeRay); series `kube_pod_status_phase{phase="Pending"}`, `kueue_pending_workloads`; on a running system: `sum(kube_pod_status_phase{phase="Pending"})`.
- **Knobs.** Governor: which orchestrator or governor is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** Each scheduler, queue and multi-cluster governor four ways: the organism alone, the orchestrator as shipped, Omni-Compass on the organism alone, Omni-Compass on the orchestrator.
- **What to do with it.** Only two of the four comparisons are Omni-Compass against its native (on top of the orchestrator; on the organism alone); the others are the orchestrator's own cost and are never counted as Omni-Compass's.

5c Kubernetes: official run playback

Playback of the official benchmark runs from their archived raw files: the native cluster and the Omni-Compass cluster under identical load, in 30-second bins, every file verified against its run's SHA-256 manifest.

The page, in its own words.

A real-time operations dashboard over the benchmark runs themselves (class L), not a model. Every series below streams half minute by half minute out of the raw files GitHub archived, each file's SHA-256 matched before a byte is drawn: native (the whole cluster with everything it ships with, everything but

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Omni-Compass) against Omni-Compass, the same load, the same second. The tiles, the candles, the heatmap, the bars and the governor's ticker all move together with section 5d (pick a benchmark, a run or a repetition in either place). Every word is the run's own reading, never one drawn from the stream.

Data pipeline: from the GitHub Actions run to this page

The heart: the governor's beat from this repetition's own audit, beside native's own clock

Why a heart. A supervisory governor is only alive while its loop turns: once every decision it reads the whole body, decides, writes, and proves it is still there. That turn is the heart. Each beat pumps every reading in through the plug and the decision out (omni_controller/controller.py, its main loop), and renews the governor's lease (master.heartbeat(), omnicompass/master.py). If the heart stops, its process gone or its lease older than its limit (the larger of 120 s and five beats), the watchdog (tools/omni_switch.py watchdog) walks every setting back to native: a stopped heart leaves the body exactly as native runs it. Native has no heart of its own: each of its controllers keeps its own clock (the pod autoscaler every 15 s, the firmware, the thermostats) and no beat joins them.

A healthy heart, never pushed. The beat runs at a fixed, calm rate (one decision a minute in these runs, the fast checks every 5 s between beats) and is never driven faster by the load it governs. What it costs is counted inside the body's own load, so the organism never pays for its own brain without knowing it. A decision that fails is recorded and skipped: it writes nothing, the body keeps the last settings that landed, and the next beat comes at its normal time (safe_step). Two gauges below read its health: the share of each beat spent deciding, and how close the lease came to its limit. The page calls a heart under strain when a decision takes more than half its beat, or a gap between beats reaches 80% of the lease. Past half its beat, the governor would have less than half its period left to act before the next reading. Both lines are this page's reading, not rules of the engine; the engine's own guard is the lease and the watchdog.

Response time: one candlestick per 30-second bin

Latency heatmap: every response, binned

Throughput: requests served per 30-second bin

Every repetition of this run, side by side

The governor's ticker: every decision of Omni-Compass in this repetition, newest first

Native's own controllers' ticker: every change its pod autoscaler and the cluster made in the same repetition, newest first

From the repetition's own audit.jsonl: the minute and second of the decision, the compass position (0 calm, 1 at the line), and the force on the pedal (push: toward more; lift: handing the setting back toward native). Then the machines it recommends, and the give-back verdict's state: "left native" means every setting stays

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

native's; "acting" means a trial on its own measurement. Native has no governor; its own controllers' moves are the second ticker, read from the native arm's capture.csv: the replicas its pod autoscaler holds, the pods waiting for a place and the machines ready. With Omni-Compass OFF the first ticker is empty, for nothing decides on top of native.

Benchmark status, by block

Its monitors.

Monitor 22: The official run's pipeline

The chosen Kubernetes run played back from its archived files: where every repetition stands.

- **Evidence and state.** Class L; replay.
- **Feed.** GitHub Actions API (read at the build); live: github-actions-exporter.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Gauge: which gauge of the run is shown.
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 23: The official run's heartbeat

Replicas, machines and the response line decision by decision, native beside Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** kube-state-metrics; the load generator's own histogram (the rig's loadgen); series `kube_horizontalpodautoscaler_status_current_replicas`, `kube_node_status_allocatable`; on a running system: `sum(kube_horizontalpodautoscaler_status_current_replicas)`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Gauge: which gauge of the run is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 24: The official run: response time candles

Each repetition's response time range, native beside Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen); series `http_request_duration_seconds_bucket`; on a running system: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[1m])))`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 25: The official run: the heat map

The chosen gauge across repetitions and time.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Monitor 26: The official run: paired bars

Every repetition's paired difference, native against Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 27: The official run: the repetitions

Each repetition's reading in the chosen gauge with its interval.

- **Evidence and state.** Class L; replay.
- **Feed.** the load generator's own histogram (the rig's loadgen).
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Gauge: which gauge of the run is shown; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

5d Kubernetes: per-repetition run telemetry

Per-run telemetry at one-second resolution, with the run's receipt and each repetition's score from the repository's own scorer.

The page, in its own words.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

These are the benchmark runs themselves, not a model: each three-run table's runs A, B and C, read back from the raw files GitHub archived (results/live/raw), native and Omni-Compass, repetition by repetition, played second by second. Every file the page reads has its SHA-256 computed and matched against the run's own SHA256SUMS_ALL.txt, and every Kubernetes repetition is scored by the repository's own scorer (tools/live_reps.py); where a run holds its own scored summary, the scorer's mean over its repetitions equals it (the count is below). Class L: real Kubernetes on kind, real databases and a real broker.

Native

Omni-Compass

The receipt of this run

This repetition, scored by the repository's own scorer

Benchmark status: landed, running, queued, or not yet run

Its monitors.

Monitor 28: Per-repetition run telemetry

The archived capture of one repetition second by second: machines ready, CPU used, pods pending, the autoscaler's replicas, native beside Omni-Compass.

- **Evidence and state.** Class L; replay.
- **Feed.** kube-state-metrics; metrics-server; Kepler; series `kube_node_status_condition`, `kube_pod_status_phase`, `kepler_node_package_joules_total`; on a running system: `sum(kube_pod_status_phase{phase="Pending"})`; `rate(kepler_node_package_joules_total[1m])`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Repetition: which repetition of the run is replayed; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

5e Kubernetes and cloud: the shipped controller's decision trace

The shipped controller's decision trace, replayed from its own audit log one decision at a time: its inputs, its choice and its writes.

Controls. Recorded data: the shipped controller replayed decision by decision from its own audit (class S).

The page, in its own words.

This is the one place in the repository where the brain drives the muscles decision by decision: the shipped controller, omni_controller/controller.py, on the toy cluster of pilot/selfpilot.py, wired as the repository wires it. Each decision runs the chain left to right: the senses, the six-state brain, the nerves and the machine organ’s grant, the compass law on the service (cloud setting), the verdict’s trial, the release gate, the plug. Every value below is the controller’s own audit record of that decision, recorded by docs/engine/export_controller_trace.py on runs A, B and C (six hours each, a decision every 60 s, class S).

Native

Omni-Compass

This decision, brain to muscle, as the controller audited it

Three runs, A B C (the whole six hours of each)

The paired change per 15 s tick, averaged over ten-minute blocks, with its 95% interval (Student’s t) in each run; GAIN or LOSS only when all three runs point the same way with every interval clear of zero (tools/readings.py). Machines and power: a fall is a cut, a GAIN; a rise is power given. Our own models (class S): a model’s small numbers are the model’s.

What the real thing reads (class L: real Kubernetes, three runs each; every Kubernetes three-run table on this page, each on its newest engine)

How steadily the governor steered on real Kubernetes: its own moves on every table results/STEERING.json holds, each on its own engine, beside native's own controllers

Its monitors.

Monitor 29: The shipped controller's trace: native

The controller every Kubernetes and cloud set runs, replayed decision by decision from its own audit: native's cluster.

- **Evidence and state.** Class S; replay.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_nodes_observed`; on a running system: `omni_nodes_observed`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 30: The shipped controller's trace: Omni-Compass

The same, Omni-Compass's cluster: every write, every trial, every hold.

- **Evidence and state.** Class S; replay.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`, `omni_writes_total`, `omni_verdict_state`; on a running system: `omni_recommended`; `increase(omni_writes_total[1h])`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

Monitor 31: The shipped controller's trace: the strip

The compass position, the force and the writes over the run.

- **Evidence and state.** Class S; replay.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`; on a running system: `omni_compass_position`; `omni_compass_force`.
- **Knobs.** Run: A, B or C (or the archived run) the monitor plays; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

native at once; every Omni-Compass side follows native (white).

- **How to read it.** A Kubernetes run played back from its own archived files, or the shipped controller replayed decision by decision from its own audit: the machines, the replicas, the response line, native beside Omni-Compass.
- **What to do with it.** Pick the run (A, B or C) and the gauge; read the paired difference in each repetition before the table's word. A run of the engine that stands carries omni-v12; an older engine's run is its own record.

5f Hardware: the GPU card and the CPU host

The highest-value integration point is the silicon itself, the chip at the far end of every stack: there the max rides each card's own limit, and every watt it takes is power given, shown beside the work it buys. Below: the slots for the metered GPU and CPU runs, every hardware run GitHub has started, then the models of one GPU card and one Linux host, native beside Omni-Compass.

Controls. Recorded data: GitHub's chip runs, then the chip models replayed from the repository's own run (class S); pick a setting in the table below.

The page, in its own words.

Each stack is run twice on the same load and the same seed: native first, then Omni-Compass. Left is native, right is Omni-Compass, and every table gives both columns and the difference. Each stack is its own model in the repository, run exactly as the repository runs it and recorded second by second by docs/engine/export_scenarios.py (class S, our own models). The clouds are section 5a and Kubernetes sections 5c to 5e, above this one; every muscle of the catalog, the seven organisms and the three-run tables are in section 4.

Hardware runs: the slots for the metered GPU and CPU runs

The GPU runs on Lambda (class P: the card's own meter) and the CPU on its own machine (class P: the package's own energy counter, tools/run_cpu_power.py) are the most valuable receipts this engine will carry: the same work for less power on the chip itself. Their omni-v12 runs are not hooked up yet: each place is written here as the repository holds it and fills in, played back from its raw files like every Kubernetes run, the moment its three runs land. Until then the views below are the repository's own models of one GPU card and one Linux host (class S), run native and then with Omni-Compass on the same load and seed.

The GPU card, metered: the one real run, replayed second by second (class P, the card's own meter)

The only run in the repository where a real card's own meter recorded native beside Omni-Compass: an NVIDIA A10 on 2 October, ten paired repetitions of 600 s an arm, native (the card as shipped),

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Omni-Compass watching only (the switch OFF: it reads and writes nothing) and Omni-Compass governing the power limit, every sample of nvidia-smi (200 ms) and every write of the governor's own audit (results/gpu/run-20261002T082232Z, docs/engine/export_gpu_run.py). It ran the card controller of commit c908054, replaced since: its rows stand as that controller's record, every one shown, until the omni-v12 run on Lambda replaces it (one exact commit, the founder's order). Its own table, re-read by the one key:

The CPU host: built and waiting for its machine (a model of a machine until then, class S)

The CPU power harness (tools/run_cpu_power.py) puts Omni-Compass on top of the Linux kernel's own frequency governor and meters the processor's package energy (RAPL). No GitHub runner exposes that counter or a writable ceiling, so its real run waits for a Linux machine that does (the register's row 79, cpu-power.yml). Until then this monitor plays the harness on its own modelled machine (--plant sim, written for its tests; results/sim/cpu_power_model, three repetitions): the mechanism moving, every decision and write of its own audit, never a measurement. The machine layer enters the whole organism as exactly 1 until the real run lands.

Every chip run GitHub has started, newest first

Where the mechanism moves on the chip models: every setting, three runs each

Read from the replayed record itself, second by second, in each of the three runs (seeds as below). A row with no second held off native is Omni-Compass deciding to stay native: its trial of one step read LOSS and was refused, or the knob is already at its limit on native.

Native

Omni-Compass

What Omni-Compass did, as its own audit records it

Three runs, A B C (the whole run of each)

Its monitors.

Monitor 32: The GPU card, metered: the one real run

The NVIDIA A10 run of 2 October replayed from the card's own meter: draw, limit, clock and temperature, requests served and the 95th percentile, native beside Omni-Compass governing (or watching only), every write of the governor marked.

- **Evidence and state.** Class P; awaiting its run: its omni-v12 run on Lambda (lambda-gpu.yml, one exact commit): the replaced controller's run stands until then, shown as such.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Feed.** NVIDIA DCGM exporter; series `DCGM_FI_DEV_POWER_USAGE`, `DCGM_FI_DEV_POWER_MGMT_LIMIT`, `DCGM_FI_DEV_SM_CLOCK`, `DCGM_FI_DEV_GPU_TEMP`; on a running system: `DCGM_FI_DEV_POWER_USAGE`; `DCGM_FI_DEV_POWER_MGMT_LIMIT`; `DCGM_FI_DEV_SM_CLOCK`.
- **Knobs.** Repetition: which repetition of the run is replayed; Arm: which arm stands on the right: Omni-Compass governing, or watching only; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The card's own meter, second by second: its draw, the power limit Omni-Compass wrote (dashed), its clock, the 95th percentile against the response line, its temperature; every write of the governor a red tick.
- **What to do with it.** Power on a card is a resource: a fall is a cut, a rise power given; the response time is the judge. The one real run on the page is the replaced controller's record and is shown as such until its omni-v12 run lands.

Monitor 33: The CPU host, metered: awaiting its machine

The CPU power harness on the host: the frequency ceiling, the clock run at, the package power and the response line, native beside Omni-Compass; today its own modelled machine (class S), the real run the day a machine with its meter runs it.

- **Evidence and state.** Class S (model) until the run; awaiting its run: a Linux machine whose processor exposes its energy counter (RAPL) and a writable frequency ceiling (register row 79, cpu-power.yml).
- **Feed.** node_exporter (rapl, cpufreq collectors); Kepler; series `node_rapl_package_joules_total`, `node_cpu_scaling_frequency_hertz`, `node_cpu_scaling_frequency_max_hertz`, `kepler_node_package_joules_total`; on a running system: `rate(node_rapl_package_joules_total[1m])`; `avg(node_cpu_scaling_frequency_hertz)`.
- **Knobs.** Repetition: which repetition of the run is replayed; Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The processor's frequency ceiling (the knob), the clock the kernel's own governor ran at under it, the package power and the busy share, native beside Omni-Compass.
- **What to do with it.** Until a machine with its energy counter runs it, the monitor plays the harness's own modelled machine: read the mechanism, never the numbers.

Monitor 34: The chip and the host, modelled

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The repository's own models of one GPU card and one Linux host, native beside Omni-Compass, second by second (class S).

- **Evidence and state.** Class S; model.
- **Feed.** NVIDIA DCGM exporter; node_exporter; series `DCGM_FI_DEV_POWER_USAGE`, `DCGM_FI_DEV_SM_CLOCK`, `node_cpu_scaling_frequency_hertz`; on a running system: `DCGM_FI_DEV_POWER_USAGE`; `DCGM_FI_DEV_SM_CLOCK`.
- **Knobs.** Stack: the GPU card, the Linux host, Kubernetes or the cloud; Run: A, B or C (or the archived run) the monitor plays; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The card's own meter, second by second: its draw, the power limit Omni-Compass wrote (dashed), its clock, the 95th percentile against the response line, its temperature; every write of the governor a red tick.
- **What to do with it.** Power on a card is a resource: a fall is a cut, a rise power given; the response time is the judge. The one real run on the page is the replaced controller's record and is shown as such until its omni-v12 run lands.

Part 6 of the page: The organism, live: native and Omni-Compass

6a The organism, live: native and Omni-Compass

The system rendered as an organism: each node is a muscle (one controlled actuator) and each ring its operating range. The left instance runs under its native controllers alone; the right runs the same controllers with Omni-Compass's brain, nerves, heart and immune system supervising them. Evidence class S: our own models, shown beside the real runs and never presented as proof.

Controls. Live: the rings, the immune system and the anatomy respond to the console.

The page, in its own words.

Every view is run twice on the same body, the same load and the same seed: native (left), then Omni-Compass (right). "One organism" is the sixty muscles in one body: the whole native organism on the left, the same organism with Omni-Compass on the right; "each muscle alone" runs each muscle native and then with Omni-Compass, one at a time. The counters under each ring and the scoreboard (section 8b) give both arms and the difference. Where the two sides look the same, they are the same. Under the max a muscle is moved only where it has room above native. A muscle already at its limit on native (— of these sixty) is left native by design. Inside one organism the bone, the power and cooling the whole body shares, holds most moves back. A dashed white ring marks every muscle that differs between the two sides at that moment; the counter under each ring counts them. In these harness views the max acts muscle by muscle

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

and the brain at the center reads the body without steering it, as in the realms harness itself; section 5e shows the live controller, where the brain's rules run every decision.

Native

Omni-Compass

The body scan: every organ, native and Omni-Compass, live from the official run playing

As a scan reads a living body organ by organ, this reads the system playing in section 5c (the benchmark, run and repetition chosen there), organ by organ, half minute by half minute, from its own SHA-256-checked files: native beside Omni-Compass. Where native has no such organ the cell says so.

The immune system, live: native beside Omni-Compass (the immune system is Omni-Compass's; native runs on its own controllers)

The body keeps itself healthy the way a human body does. Every room and every machine pool has a native twin: the same muscle under its own controller, on native's heat and load, the body's memory of healthy. A room that stands warmer than its twin is inflamed. Where its own knob can cool it, it is cured on the spot. Where it cannot, it sends the whole body a signal, its heat debt, like a cytokine calling for help. The other muscles answer: the machine pools give back what Omni-Compass added until the room stands no warmer than native's. The skeleton (the bone: the power and cooling lines the whole body shares) grants nothing it cannot carry. A cooling setting is walked back, never snapped. A pool never loses a machine its queue needs, a battery is timed for its line, and no sense is locked. On the ring a pulsing red circle marks the inflamed muscle, and red signals run from it to every machine pool that answers it.

The anatomy, part by part, in the order of control: the brain first, the whole organism last

Its monitors.

Monitor 35: The organism, live: native

Sixty muscles of the stress model, each on its own native controller, at their positions in the band.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); kube-state-metrics; series `omni_recommended`, `kube_horizontalpodautoscaler_status_current_replicas`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused; Body: which body the organism runs: the harness, the organism, every muscle alone; Muscle: which muscle's own card is read.

- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

Monitor 36: The organism, live: Omni-Compass

The same sixty muscles with Omni-Compass on top of their native controllers, the brain's grants and pushes.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`, `omni_recommended_moves_total`, `omni_writes_total`; on a running system: `omni_recommended`; `increase(omni_writes_total[1h])`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused; Body: which body the organism runs: the harness, the organism, every muscle alone; Muscle: which muscle's own card is read.
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

Monitor 37: The immune system

Where the two sides differ, and the body's heat debt: a room warmer than its native twin, inflamed, given back.

- **Evidence and state.** Class S; live.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Feed.** Prometheus Alertmanager; tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `ALERTS`, `omni_compass_decisions_total{zone="wall"}`; on a running system: `sum(ALERTS{alertstate="firing"})`; `increase(omni_compass_decisions_total{zone="wall"}[1h])`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

Monitor 38: The swarm

The robot swarm's missions, native beside Omni-Compass with the proximity reflex.

- **Evidence and state.** Class S; live.
- **Feed.** the swarm runner's own log (tools/run_swarm.py); no Prometheus feed.
- **Knobs.** Speed: 1×, 4×, 16× or 60× of the engine's clock; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** The organism the brain governs, muscle by muscle: positions in the compass band, the brain's grants and pushes, the immune system's heat debt; native's own controllers under every muscle.
- **What to do with it.** Turn the console's knobs (demand, noise, sensor delay, faults) and watch both arms take the same input; the difference between the lanes is what Omni-Compass did.

6b Telemetry analytics and Omni-Compass in its own words

The same telemetry analysed four ways: a three-dimensional tensor view, a difference matrix, a principal-component decomposition and Omni-Compass's own account of its run; the service level and error budget in SRE terms; and Omni-Compass answering the viewer's questions about itself from the page's own records.

Controls. Recorded data: the official run chosen in section 5c (class L). Console controls never modify it.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The page, in its own words.

The engine's own record is a tensor: two arms × every decision × sixty muscles, each entry a muscle's compass position (0 calm, 0.5 the center of its band, 1 its line), recorded from realms/harness.py by docs/engine/export_harness_trace.py (class S, our own models). It fills in below decision by decision as section 6a plays, and the same numbers are read three more ways: the difference between the arms, the body's principal motions, and Omni-Compass's own account of what it is doing. Then the official runs (class L) are read as an SRE reads a service: the service level, the error budget burned and every gauge against native; and Omni-Compass answers questions about itself, reading the page's own records through tools and turning the dashboard to a run. No word on this panel is drawn from a picture: every GAIN, EVEN, LOSS and power given is a run's own reading.

Omni-Compass, in its own words

Omni-Compass speaks for itself here: every line is read from its own records at the moment on screen (the official run playing in section 5c, its governor's own audit, and the engine's harness on the console's clock), and says what it is doing, beside what native does. Speak aloud reads it out on the viewer's own browser voice as the run plays, and reads its answers below too.

The telemetry tensor, in three dimensions

The difference matrix: Omni-Compass against native

The body's principal motions (the tensor's eigen-decomposition)

The service level and the error budget, this repetition

Every gauge of this repetition against native

Ask Omni-Compass: it answers for itself, from its own records on this page

Ask in your own words, or pick a question. Omni-Compass answers in the first person, for its own mechanism and its own runs, reading only this page's records through six tools. The words are composed for it by Claude on claude.ai, on the viewer's own account and with the viewer's permission; opened anywhere else, the rest of the page runs whole without it.

Connecting…

Omni-Compass's answers are drawn from this page's records and are not reviewed by any outside party. © 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Its monitors.

Monitor 39: Telemetry tensor: the cube

The organism's telemetry as a tensor (muscle × gauge × time), native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** kube-state-metrics; the service's own metrics.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

Monitor 40: Telemetry tensor: the difference

The difference of the two sides' tensors, cell by cell.

- **Evidence and state.** Class S; live.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

Monitor 41: Telemetry tensor: the modes

The principal modes of the telemetry (its matrix decomposed).

- **Evidence and state.** Class S; live.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

Monitor 42: Service level: the objective

Time inside the response line against the objective, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** SLO recording rules (sloth, pyrra) over the service's histogram; series `slo:sli_error:ratio_rate5m`; on a running system: `1 - slo:sli_error:ratio_rate5m`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

beside Omni-Compass.

- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

Monitor 43: Service level: the error budget burn

How fast each side burns its error budget.

- **Evidence and state.** Class S; live.
- **Feed.** SLO recording rules (sloth, pyrra); series `slo:error_budget:remaining`; on a running system: `slo:error_budget:remaining`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

Monitor 44: Telemetry radar

Every gauge family on one radar, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** the build (the repository's tables, no live feed: a recording rule would compose the layers' ratios).
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **How to read it.** The organism's telemetry as a tensor (muscle by gauge by time), its difference between the arms, its principal modes, the service level against its objective and the error budget burned, native beside Omni-Compass.
- **What to do with it.** The service level and the budget are what an on-call engineer is paged on: watch the burn rate on both arms under the same knobs.

7 Control console: every runtime control of the live engine

The engine these knobs turn is the engine's own mathematics running in this page: the brain (omnicompass/core.py, integrated by RK4), the compass law (omnicompass/compass_law.py) and the verdict (omnicompass/verdict.py). docs/engine/check_live_engine.py proves the port equal to the Python and the C++ on every build. It governs the stress-test model's sixty muscles (class S), each on top of its own native controller. Every knob acts on both sides at once, the same demand, the same draw of noise, the same lost pool, so the one difference left between them is Omni-Compass. The knobs move the live engine in sections 6a and 8a to 8c, and the kill switch and the frame load also move this page's own renderer (section 11). A recorded run (the benefit board, the chips' and Kubernetes' replays, the official runs) is a record: no knob rewrites it.

The page, in its own words.

Do no harm. Every control here acts on both arms alike; what Omni-Compass does with them is held by its guards: no muscle driven below native or past its own limit, nothing taken from one muscle to give to another, a give-back tried against native first and refused on a LOSS. Its one meaning is written once, in the back matter (iv) as DISCLOSURES.md section 3 writes it.

What these controls do. Runtime parameter control and fault injection on the live engine: load scaling, noise injection, sensor latency, capacity loss, a demand surge, a sensor blackout, a security hold, the dial, and the kill switch. Every control is applied to both arms identically; the response is plotted per decision, native beside Omni-Compass.

System response, per decision

Frame timing: the inter-frame interval against the 60 Hz budget

Control event log, newest first

Control-loop performance assessment, this day (live)

Its monitors.

Monitor 45: The console's response

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The live engine's response to every knob of the console, plotted per decision, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused; Pause: hold the engine's clock; Frame load: +0, +8 or +16 ms burned inside every frame of this page.
- **How to read it.** The live engine's response to every knob, native beside Omni-Compass, and the page's own frame budget.
- **What to do with it.** Every knob acts on both arms alike; the difference between them is Omni-Compass's doing alone.

Monitor 46: The frame budget

The time between frames of this page against its line under the frame load knob.

- **Evidence and state.** Class S; live.
- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Knobs.** Frame load: +0, +8 or +16 ms burned inside every frame of this page.
- **How to read it.** The live engine's response to every knob, native beside Omni-Compass, and the page's own frame budget.
- **What to do with it.** Every knob acts on both arms alike; the difference between them is Omni-Compass's doing alone.

Part 8 of the page: Control architecture: the brain

8a One control cycle, traced step by step

One control cycle traced end to end at reduced speed: the sensed inputs, the brain's decision, and the actuator (muscle) it wrote. Evidence class S: our own models, shown beside the real runs and never presented as proof.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Controls. Live: the cycle and the controls below share the console's state.

The page, in its own words.

One decision, in the order the code runs it

Controls

The max: every muscle to its own limit, never below native; the capacity it holds is power given. Tune-down: never more than native, and a step back only with the organ’s grant and the release gate. In this model native already sits near its band, so the tune-down finds little to hand back; the scoreboard (section 8b) reads its runs A, B and C live when the dial is on the tune-down. The spread of positions can widen a little, because the calm muscles it hands back move toward their line while the busy ones stay.

Each stress lasts 40 decisions from the moment you press it. Both arms meet the same demand.

Its monitors.

Monitor 47: One control cycle, traced

The compass band with its cushions, the two forces and one decision, step by step.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`; on a running system: `omni_compass_position`; `omni_compass_force`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

8b Phase plane and scoreboard

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The phase-plane state of every muscle (its position and velocity in its band) for both arms at the same instant, with the running scoreboard. Evidence class S: our own models, shown beside the real runs and never presented as proof.

Controls. Live: every plane, strip and well responds to the console.

The page, in its own words.

How to read the plane. It is a phase portrait, the picture control engineers use for a system's state. Across is where each muscle sits in its band: p is 0 when calm, 0.5 at the center and 1 at its line. Up and down is how fast it moves there (v, per decision). A muscle at rest sits on the middle line. One climbing toward its line moves up and right, and past the right edge it is past its line. Native is on the left and Omni-Compass on the right: the same muscles at the same instant, one dot each. In the realms harness each muscle answers to its own rule, so no single field is drawn. In the stress-test model the shading on the right is the force the compass law would apply there: the add force gives capacity and the hand-back force returns it. Past the 0.95 wall the law fails up at once. Under each plane, the same instant in words.

Native

Omni-Compass

Live stat panels: native beside Omni-Compass, now and over the last 120 decisions

Each panel is one gauge of the live organism, read every decision on both sides, as an operations dashboard shows it. It gives native's value now and Omni-Compass's, the last 120 decisions of both (native solid amber, Omni-Compass blue), and the running paired reading in the tables' words. With the master switch OFF the decisions are shaded red and Omni-Compass's side follows native: its value equals native's and its line is drawn in native's colour, dashed on top of native's, so both stay in view.

Native against Omni-Compass

How each row is read: the paired change, native against Omni-Compass at the same decision, averaged over blocks of ten decisions so neighbouring decisions do not count twice, with its 95% interval (Student’s t over the blocks). This run: an interval that includes zero reads EVEN; clear of zero, the word follows the gauge’s aim (work: a rise is GAIN; misses, failed work, waiting: a fall is GAIN; energy or capacity held: a rise is power given, a fall a cut). Three runs (tools/readings.py): GAIN or LOSS only when all three runs point the same way with every interval clear of zero; EVEN when an interval includes zero; “the runs disagree” when clear runs point different ways. The spread of positions and the settings are shown, never judged: no rule of the engine aims at them. Run and scored by The Omni-Compass LLC's own code. No outside party has reviewed or approved these results.

Its monitors.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Monitor 48: Phase plane: native

Native's muscles in position and velocity.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

Monitor 49: Phase plane: Omni-Compass

Omni-Compass's muscles in position and velocity, the band's walls drawn.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

Monitor 50: The scoreboard strip

Breaches, writes, holds and the brain's calm over the day, native beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_decisions_total`, `omni_writes_total`; on a running system: `increase(omni_compass_decisions_total{zone="wall"}[1h])`; `increase(omni_writes_total[1h])`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

8c The brain's internal state

The brain's six state variables rendered on their potential-energy landscapes: a state resting at a minimum is stable; a state driven toward a maximum is under load. Evidence class S: our own models, shown beside the real runs and never presented as proof.

Controls. Live: every plane, strip and well responds to the console.

The page, in its own words.

The same state the cycle panel prints, drawn as the physics it is. Each ball is a live value of the engine's brain, integrated by RK4 every decision. In the stress-test model its push and target feed the body's levers. In the harness views the harness arm (realms/compass_arm_v5.py) acts muscle by muscle with no brain, so the brain drawn here is the page's own, fed the harness's readings. With each muscle alone there is no body, and its power and thermal readings are held at 0.3. Native has none of this: no state, no memory of

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

need, no stress, no tide; each controller sees only its own reading, one to three decisions late.

U on the double well · eq. (2)

$V(U) = -\mu(U^2/2 - U^4/4) + (\dot{E}/E_{max})\cdot U$. A rising E tilts the well toward the basin of collapse.

S in its potential Φ · eqs. (5)–(6)

$\Phi(S) = \alpha_s S^2/2 + \beta_s S^3/4 - \delta S$. S rolls to S*; the senses pull it back each decision.

The bath B · eq. (7)

$\dot{B} = \gamma_c \delta S - (\omega_B/Q_B)\dot{B} - \omega_B^2 B$. The slow tide of physical load.

Native, the same decisions: what its own controllers read and do

Native has no brain to draw: each of its controllers reads one reading, its own muscle's, and moves. Drawn live from the same decisions as the wells above: the mean position its muscles' readings put them at (0 calm, 1 at the line), the share of its muscles past their line, and how many of its own controllers moved their setting this decision. Class S, our own models.

The nerves: four scalars multiply into calm

Native has no nerves: nothing grants or refuses a controller, and a blind sense or a security hold changes nothing it does.

calm = $\kappa\cdot h\cdot(1 - (\sigma - 1)\quad)\cdot(1 - \min(\nu, 1))$. An organ may hand capacity back only when calm ≥ its θ, the line is clean and every sense is live; expanding stays allowed except under a security hold. Under the max, lifting off toward native never needs this grant; going below native does.

Its monitors.

Monitor 51: The brain's state: the well

The brain's double-well potential and where its state sits.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

16× or 60× of the engine's clock; One decision: step the engine one decision while paused.

- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

Monitor 52: The brain's state: the six variables

E, U, I, S, B and C integrated by RK4 every decision.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_compass_position`, `omni_compass_force`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

Monitor 53: The brain's state: the organs

Each organ's grant to expand or contract, decision by decision.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_recommended`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.

- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

Monitor 54: Native's own controller beside the brain

What native's controller would do at the same decision, beside the brain's grant.

- **Evidence and state.** Class S; live.
- **Feed.** kube-state-metrics; series `kube_horizontalpodautoscaler_status_desired_replicas`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.
- **How to read it.** The brain's six state variables integrated by the fourth-order Runge–Kutta step every decision, the compass band with its cushions and its two forces, the organs' grants, native's own controller beside each grant.
- **What to do with it.** The brain never forces: it reacts to what the body shows. A grant to expand is not a write; a write happens only where the muscle's own trial allows it.

8d The verdict: change acceptance testing

The change-acceptance test. Each step is trialled against native and kept unless the measurement reads LOSS or its estimated cost exceeds the 2% allowance. An EVEN result (no difference resolvable from the noise) is also accepted, so on high-variance workloads a step with a small true cost can pass; the lab below measures that false-acceptance rate.

The page, in its own words.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Before a knob moves one step past native, the step runs as a trial: costs at native first, then at the step. omni-v12 reads the change exactly as a report reads a row, on the log of each cost, with its 95% interval. Set what the step truly costs and watch the rule decide. It opens wired as the shipped Kubernetes controller is wired, on the noise the real runs measured.

This trial

Over many trials at this cost

Native: its knob never moves past its own setting, so there is no trial and nothing to judge; every step Omni-Compass takes past native has to pass this test first. The medians alone allow a step when the trial's median is within the allowance of the reference's. omni-v12's rule also refuses any step whose interval sits wholly above zero. It measures on while the interval is wider than the allowance, up to three times the pieces measured each side. It keeps a step only if both estimates are inside the allowance. Ported from omnicompass/verdict.py; the costs are modelled (class S).

How each muscle wires it. The Kubernetes controller measures 200 requests each side against a 2% allowance (omni_controller/controller.py --verdict-samples 200 --allow 0.02), the cost being each request's response time. The noise is measured at every build on each request of the newest tables' own runs. The σ 0.05 and 0.25 this page carried before 19 October did not come from these files and are withdrawn. The GPU card measures 30 requests (gpu_compass.py --verdict-samples 30); in its model the card's own time per request carries no noise. The toy gate measures 60 (tools/toy_gate.py, σ 0.25). What the rule does there is computed live in this page as you read: trials at each cost with this lab's own rule, 200 requests a side to start (the trial side measured on to 600 while its interval is wider than the allowance), a 2% allowance and the measured σ (modelled, class S). With noise this wide, 200 requests a side do not refuse a small LOSS on their own; with no noise any LOSS it can read is refused, even 0.1%. A larger sample would refuse more, and any change to the rule or its sample is the next engine version, landed through the toy gate and told to the founder (his order of 19 October). Above the verdict the max never goes below native at all, so in the default it never runs (section 5e: no trial on top of the HPA).

Its monitors.

Monitor 55: The verdict: one trial

A give-back trial judged against native: its reference, its step, its interval and the word.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`; on a running system: `omni_verdict_state`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

limit) or the tune-down (never more than native); Demand: ×0.25 to ×2 of the day's own demand, both arms alike; Noise: σ of the demand's noise, one draw a muscle a decision, both arms alike; Sense delay: decisions added between what a muscle does and what its controller reads, both arms alike; Faults: a surge, half the machines lost, a sense blind, a security hold, on the chosen body part; Speed: 1×, 4×, 16× or 60× of the engine's clock; One decision: step the engine one decision while paused.

- **How to read it.** A give-back trial judged against native: its reference, its step, its interval, the word; then the keep ratio at every true cost.
- **What to do with it.** A step is kept only when its interval stays inside the allowance; a refused step leaves the muscle where native had it.

Monitor 56: The verdict: every true cost

The keep ratio of the verdict at each true cost, from trials run in this page's idle time.

- **Evidence and state.** Class S; live.
- **Feed.** tools/omni_exporter.py (the governor's own audit, Prometheus text format); series `omni_verdict_state`.
- **Knobs.** Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white); The dial: the max (every muscle to its own limit) or the tune-down (never more than native).
- **How to read it.** A give-back trial judged against native: its reference, its step, its interval, the word; then the keep ratio at every true cost.
- **What to do with it.** A step is kept only when its interval stays inside the allowance; a refused step leaves the muscle where native had it.

9 Failure modes, controls and residual risk

The failure-mode register: every known way Omni-Compass can fall short, the control that mitigates it, that control's coverage, and the residual risk where it does not reach. Every failure mode the live engine can be put through has a fault-injection control; trigger it and observe both arms on the console. Below: the resource costs the three-run tables report but do not judge.

Controls. Live: every way below that the live engine can be put through has its own button, and the response plays on the console (section 7), native beside Omni-Compass.

The page, in its own words.

Failure-mode register: control, coverage and residual risk

Reported, unjudged metrics: the resource cost of the work

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Its monitors. This section draws no canvas: its views are tables and text the page builds from its records.

10 Component specification and mathematics

The component specification: every part, its equations and the source module that implements it, in Unicode as the repository writes them.

Controls. Live: the brain's six state variables below are read from the live engine every decision, beside the equations that evolve them.

The page, in its own words.

Items 1 to 5 and 8 are the engine’s code and run in every view. Items 6 and 7 as written (the compass law on each muscle, the plug’s lever) drive the stress-test model. In the harness views the max and the plants’ own overrides take their place. The live controller runs the compass law on the service, as section 5e shows.

The stress-test model, written out (class S, drawn for this page)

Who drives what

What is the engine's code

The six-state engine, its constants and RK4 integrator, the assimilation, push and ρ*, the four nerve scalars and the organ thresholds, the compass law and the verdict in words are ported from omnicompass/core.py, adapter.py, nervous_system.py, compass_law.py and verdict.py at commit c8ba7775d. All are inside the omni-v12 fingerprint. omni-v9 added omnicompass/student_t.py, and the page carries its Student t too, computed by bisection on its own distribution and rounded to three decimals as the printed tables are. omni-v10 changed only the CityLearn wire's glide, omni-v11 only the Kubernetes governor's blind-reading guard, and omni-v12 only that governor's hold on a refused raise. Checked against the Python on seeded inputs (docs/engine/check_live_engine.py): 8,000 decisions of the governor (state, push, ρ*, κ, h, σ, ν, calm), 48,000 organ grants, 4,800 compass forces, 300 verdict intervals and 400 full verdict trials. A build passes only when the largest difference is under 10^{-12}, no grant and no verdict differs, and the Student t is equal on all 500 degrees of freedom checked. And against the C++ twin of the core (cpp/src/core.cpp, compiled by the check with docs/engine/cpp_core_probe.cpp): 2,000 seeded states, the derivative and the RK4 step within 10^{-12} of each other. The check prints its own measured differences in each build's report on GitHub. The page's engine, the Python and the C++ are one engine. The harness views are not ported: they are the harness’s own record, made by its code.

The two bodies

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The realms harness (the default). The sixty muscles are one organism of realms/harness.py. Each is a plant of realms/plants.py: machine pools under their autoscalers, cooled zones, batteries on their lines, motion axes and process loops. They are coupled through their heat and their power, under the climb (1 2 3 2 3 4 …), for 240 decisions of 15 s. Omni-Compass is the max of realms/compass_arm_v5.py with the body's bone, its twins and the muscles' cards; at 80% the brain goes silent and the lease walks every setting back to native over 24 decisions. Recorded on seeds 3000, 3001 and 3002 (runs A, B and C) by docs/engine/export_harness_trace.py; the page replays the record. Each muscle alone is the same harness one muscle at a time, as run_muscle_arm runs the preregistered muscle table: under the line (work waiting at the start), the max with the muscle's own twin, and the lease after 80%. Each muscle's decisions are laid on 240 frames. A frame shows the last decision it covers, and reads over the line if any of them was. In that harness the brain's state is computed from the body's aggregate, as a governor reads it, and shown; the max itself acts muscle by muscle. The stress-test model. A model drawn for this page so the body can be stressed by hand (a surge, a blind sense, a security hold, the kill switch), evidence class S; like every view it runs native and Omni-Compass, side by side, on the same demand. Sixty muscles named from the 1,107-muscle catalog (twelve each from the spine and the four realms), each a capacity lever with its own response time τ and sensing delay, under an HPA-style native controller (target 0.70, 10% tolerance, five-decision scale-down window). Demand climbs 1 2 3 2 3 4, surges, eases and idles over six hours, the same for both arms. The lever rules that turn F into a write are this page's, written out above and kept inside the doctrine. Under the max: never below native, and lifting off hands back toward native. On the tune-down: never above native, and below it only with the organ's grant and the release gate.

Its monitors. This section draws no canvas: its views are tables and text the page builds from its records.

11 Self-governed rendering: Omni-Compass inside this page

The page's own render loop is governed by the same compass law, measured on the client device, native beside Omni-Compass.

The page, in its own words.

The engine that runs this page has Omni-Compass in it. Native is the browser's own loop: every chart on screen drawn on every frame, and when the frame overruns, whole frames dropped. Omni-Compass is the same loop with the engine's own compass law governing it (omnicompass/compass_law.py, ported here and checked against the Python). Each chart is a muscle whose knob is how many frames it waits between draws, and the reading is the time between frames. Below the band's center the pedal lifts and every chart is handed back to native, drawn every frame. Toward the line it pushes: the charts farthest from where you are looking wait a frame or more, so the frame itself stays inside its line. Every step is a trial run to its full measurement and kept only when the frame shortens; a step that buys nothing is refused and every chart handed back to native (the reflex rule: where a knob cannot bring a GAIN, it stays native). The reading is

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

the time between frames, the band two frames wide with its center one frame at 60 Hz. The two arms take turns in paired windows of three seconds on this same page, on your own machine, and are read by the three-run rule's interval (class L: this browser, live; a reading of this page, never a benchmark of record).

Native

Omni-Compass

Frame timing: the inter-frame interval against the 60 Hz budget

Paired measurement of this page, per window

Its monitors.

Monitor 57: Self-governed rendering

This page's own render loop governed by the compass law: the frame time against its line, native (no governor) beside Omni-Compass.

- **Evidence and state.** Class S; live.
- **Feed.** this page's own clock (no exporter: the page measures itself).
- **Knobs.** Frame load: +0, +8 or +16 ms burned inside every frame of this page; Kill switch: Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white).
- **How to read it.** This page's own render loop governed by the compass law: the time between frames against its line, native (no governor) beside Omni-Compass.
- **What to do with it.** Load the page with the frame-load knob and watch the governor shed chart frames to keep every frame inside its line.

v Disclosures, governing law and contact

The page, in its own words.

What this page is

Disclosures

Contact

Its monitors. This section draws no canvas: its views are tables and text the page builds from its records.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Part III. Operating it as the engineer in charge

Watching: the first minute and the first hour

- **The first minute.** Read the switch (ON green, or engaged red with OFF white). Read the status board's tiles: whether Omni-Compass is engaged, which signals are out of tolerance. Read the alarms, worst first: every LOSS row of every table is raised there with its source record.
- **The first hour.** Watch the golden signals of the live organism (errors, throughput, latency, saturation), native beside Omni-Compass. Turn the console's knobs one at a time (demand, noise, sensor delay, a fault) and read how both arms take the same input. An alert minute is a whole minute whose 95th percentile, failed requests counted over, was over the line: a minute an on-call page would be open. Omni-Compass aims at none: an alert minute on its arm is found and fixed, never accepted.
- **No babysitting tax.** Omni-Compass is an autopilot: nobody turns a knob for it and nobody checks its work by hand. Every write is read back, every give-back runs on its own trial, every reason is in its audit; the settings it wrote on its own and its additions made inside the line are shown on every Kubernetes and cloud receipt.

When a row reads LOSS

A LOSS row is never cut and never tuned away. The rule: first suspect the wiring, the native setup or the scoring; find where the mechanism or its wiring is wrong and correct that, in the toy gate first (`tools/toy_gate.py`), never in a real run; a change that reads no LOSS in the gate lands as the next engine version and every benchmark runs again on it. Where no setting of the muscle's knob brings the row EVEN, the muscle stays native on that stack: if Omni-Compass would hurt something, no mechanism of Omni-Compass is pushed through that muscle. The status board's table of LOSS rows names, for every one, whose row it is (Omni-Compass against its native, or an orchestrator's own cost with no Omni-Compass in either arm), its cause as the repository records it, and the knob that answers it; Appendix C lists them all.

Tuning and tweaking

- **The dial.** The max, the default brain: every muscle runs at its own limit, never past it and never below native. The tune-down: never more than native; capacity handed back only with the organ's grant and the release gate. In the cloud every set runs at the tune-down on the machines alone, so the bill can only fall or stay.
- **A muscle's own card.** Every muscle is wired on its own setting (`realms/muscle_cards_v5.csv`, the engine's setting otherwise): its band, cushions and glide. The factory tune is the owner's; a licensee

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

may retune, add on or tune down any muscle, and a changed tune's results are its own (`DISCLOSURES.md` section 3).

- **The next version.** Any change to a rule, gain, guard, preset or muscle makes the next engine version: built on its own branch, tried in the toy gate against the engine that stands, landed with `tools/land_engine.py` when it reads no LOSS, and every benchmark run again on it.

What waits, and what fills it

- **The GPU card.** The one metered run is the replaced controller's (2 October); the omni-v12 run on Lambda, from one exact commit given to the founder, replaces it.
- **The CPU host.** A Linux machine whose processor exposes its energy counter (RAPL) and a writable frequency ceiling (register row 79, `cpu-power.yml`); until then the monitor plays the harness's own modelled machine.
- **Azure.** The set to the paid cap runs now: 29 of 100 repetitions measured at the build, under the stopping rule (the bill read at 10, 20, 40 and 70 repetitions with 99% intervals); its table is written when the set ends.
- **Google Cloud.** the hourly gate holds
- **Amazon Web Services.** the hourly gate holds
- **The stacked run.** The whole stack in one run, Omni-Compass at every piece at once against native at every piece, preregistered in `docs/WHOLE_SYSTEM_PREREGISTRATION.md`; until it runs, section 3 is a composition.

Rebuilding the page and the book

The page is rebuilt by GitHub itself (`.github/workflows/live-engine.yml`, `docs/engine/build_live_engine.py`) whenever the engine, a model or a result changes, and every three hours; `build_live_engine.py --check` rebuilds every record in memory and fails if any differs from the page; `docs/engine/check_live_engine.py` proves the page's engine equal to the Python and the C++; `docs/engine/check_records.py` reproduces every record from the repository's own code; `tools/engine_runs.py --verify` runs the page's own loop of the Python's runs in node and compares the digests. This book is rebuilt from the page with `python3 docs/book/build_live_manual.py`.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Back matter

Appendix A. Every monitor

No.	Monitor	Section	Class	State
1	The master switch and the kill switch	1a The monitor wall and the control panel	live	live
2	The Python's runs, reproduced here	1a The monitor wall and the control panel	S (the engine itself)	live
3	The Python's 500 scenarios, replayed	1a The monitor wall and the control panel	S	replay
4	Golden signal: errors	1a The monitor wall and the control panel	S	live
5	Golden signal: throughput	1a The monitor wall and the control panel	S	live
6	Golden signal: latency	1a The monitor wall and the control panel	S	live
7	Golden signal: saturation	1a The monitor wall and the control panel	S	live
8	GitHub Actions: the runs in flight	1b CI pipeline	record	record
9	The benefit board	2 The benefit board	L/S	record
10	Where the benefit lives, gauge by gauge	2 The benefit board	L/S	record
11	The whole organism, composed	3 The whole organism, every way it is bought	S (of P and L)	composed
12	The seven organisms, three loads	4 Measured effect in full	S	record
13	Every muscle of the catalog	4 Measured effect in full	S	record
14	The scale-out grid	4 Measured effect in full	S/L	record
15	Azure: the bill, run by run	5a Cloud providers	L (metered by Azure)	record

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

No.	Monitor	Section	Class	State
16	Google Cloud: the gate and the bill	5a Cloud providers	L	awaiting its run
17	Amazon: the gate and the bill	5a Cloud providers	L	awaiting its run
18	The cloud setting, live: native's machines	5a Cloud providers	S	live
19	The cloud setting, live: Omni-Compass's machines	5a Cloud providers	S	live
20	The orchestrators' road	5b Orchestrators and governors	L	record
21	An orchestrator, four ways	5b Orchestrators and governors	L	record
22	The official run's pipeline	5c Kubernetes	L	replay
23	The official run's heartbeat	5c Kubernetes	L	replay
24	The official run: response time candles	5c Kubernetes	L	replay
25	The official run: the heat map	5c Kubernetes	L	replay
26	The official run: paired bars	5c Kubernetes	L	replay
27	The official run: the repetitions	5c Kubernetes	L	replay
28	Per-repetition run telemetry	5d Kubernetes	L	replay
29	The shipped controller's trace: native	5e Kubernetes and cloud	S	replay
30	The shipped controller's trace: Omni-Compass	5e Kubernetes and cloud	S	replay
31	The shipped controller's trace: the strip	5e Kubernetes and cloud	S	replay
32	The GPU card, metered: the one real run	5f Hardware	P	awaiting its run
33	The CPU host, metered: awaiting its machine	5f Hardware	S (model) until the run	awaiting its run
34	The chip and the host, modelled	5f Hardware	S	model

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

No.	Monitor	Section	Class	State
35	The organism, live: native	6a The organism, live	S	live
36	The organism, live: Omni-Compass	6a The organism, live	S	live
37	The immune system	6a The organism, live	S	live
38	The swarm	6a The organism, live	S	live
39	Telemetry tensor: the cube	6b Telemetry analytics and Omni-Compass in its own words	S	live
40	Telemetry tensor: the difference	6b Telemetry analytics and Omni-Compass in its own words	S	live
41	Telemetry tensor: the modes	6b Telemetry analytics and Omni-Compass in its own words	S	live
42	Service level: the objective	6b Telemetry analytics and Omni-Compass in its own words	S	live
43	Service level: the error budget burn	6b Telemetry analytics and Omni-Compass in its own words	S	live
44	Telemetry radar	6b Telemetry analytics and Omni-Compass in its own words	S	live
45	The console's response	7 Control console	S	live
46	The frame budget	7 Control console	S	live
47	One control cycle, traced	8a One control cycle, traced step by step	S	live
48	Phase plane: native	8b Phase plane and scoreboard	S	live
49	Phase plane: Omni-Compass	8b Phase plane and scoreboard	S	live

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

No.	Monitor	Section	Class	State
50	The scoreboard strip	8b Phase plane and scoreboard	S	live
51	The brain's state: the well	8c The brain's internal state	S	live
52	The brain's state: the six variables	8c The brain's internal state	S	live
53	The brain's state: the organs	8c The brain's internal state	S	live
54	Native's own controller beside the brain	8c The brain's internal state	S	live
55	The verdict: one trial	8d The verdict	S	live
56	The verdict: every true cost	8d The verdict	S	live
57	Self-governed rendering	11 Self-governed rendering	S	live

Appendix B. Every knob

Knob	What it does	Where it is
Kill switch	Omni-Compass ON (green) or engaged (red): every setting handed back to native at once; every Omni-Compass side follows native (white)	the master switch (section 1b)
The dial	the max (every muscle to its own limit) or the tune-down (never more than native)	the console's dial
Demand	×0.25 to ×2 of the day's own demand, both arms alike	the console's demand slider
Noise	σ of the demand's noise, one draw a muscle a decision, both arms alike	the console's noise slider
Sense delay	decisions added between what a muscle does and what its controller reads, both arms alike	the console's delay slider
Faults	a surge, half the machines lost, a sense blind, a security hold, on the chosen body part	the console's fault buttons

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Knob	What it does	Where it is
Speed	1×, 4×, 16× or 60× of the engine's clock	the console's time knob
One decision	step the engine one decision while paused	the console's time knob
Pause	hold the engine's clock	the console's time knob
Frame load	+0, +8 or +16 ms burned inside every frame of this page	the console's frame-load knob
Run	A, B or C (or the archived run) the monitor plays	the monitor's own buttons
Repetition	which repetition of the run is replayed	the monitor's own buttons
Arm	which arm stands on the right: Omni-Compass governing, or watching only	the monitor's own buttons
Stack	the GPU card, the Linux host, Kubernetes or the cloud	the monitor's own buttons
Governor	which orchestrator or governor is shown	the monitor's own table
Scenario	on-site bare metal, through the cloud, in the device, as a service	the monitor's own buttons
Reading	every measured piece carried, or the index's strict rule	the monitor's own buttons
The chip's run	the chip's one run beside the count or in it	the monitor's own buttons
Gauge	which gauge of the run is shown	the monitor's own buttons
Filter	which rows, words, families or organisms are shown	the monitor's own buttons
Body	which body the organism runs: the harness, the organism, every muscle alone	the organism's own buttons
Muscle	which muscle's own card is read	the organism's own table
Runs	how many of the Python's runs this page computes again	the monitor's own buttons

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Knob	What it does	Where it is
Print	the receipts of the runs, printable, with the digests and the notice	the monitor's own button
Reading direction	the wall read right to left (as set) or left to right	the wall's own button
Seed base	which held-out seed base's 500 scenarios are replayed	the monitor's own buttons

Appendix C. Every LOSS row, whose it is, its cause and the knob that answers it

5 LOSS rows are Omni-Compass against its native; 75 are an orchestrator's own cost against the organism alone, with no Omni-Compass in either arm. Every one is shown as its table writes it.

- **Omni-Compass against native.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream (omni-v12, `results/live/V12_BINPACK.md`): queue: job turnaround, submitted to done, mean (s), A B C +1.4% (+0.8 to +1.9) · +1.0% (+0.4 to +1.7) · +1.2% (+0.7 to +1.6). Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row EVEN in the toy gate, the muscle stays native on this stack.
- **Omni-Compass against native.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_TOP.md`): pod start wait, total (s), A B C +80.9% (+37.3 to +124.5) · +104.5% (+46.9 to +162.2) · +54.4% (+20.0 to +88.9). Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the test machine, not the governor: the image unpacked inside the window on the one runner every kind node shares (`docs/POD_START_WAIT.md`); the rig pre-pulls the image in both arms.
- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BARE.md`): response time (ms), 99th percentile, A B C +9.8% (+1.5 to +18.0) · +4.4% (+0.1 to +8.7) · +13.0% (+4.7 to +21.2). Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row EVEN in the toy gate, the muscle stays native on this stack.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **Omni-Compass against native.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BARE.md`): batch: queue finished (s), A B C +0.8% (+0.1 to +1.5) · +0.7% (+0.2 to +1.2) · +0.6% (+0.2 to +1.1). Cause: Amendment 10: what the counted runs of 20 October found (written after those runs, before the Gödel runs start again) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the muscle's own card (its band, cushions and glide); a setting that reads LOSS in the toy gate is refused and the muscle stays native.
- **Omni-Compass against native.** The Kubernetes scheduler as shipped under the batch stream (omni-v12, `results/live/V12_VANILLA.md`): queue: job turnaround, submitted to done, mean (s), A B C +0.9% (+0.2 to +1.7) · +0.9% (+0.1 to +1.6) · +1.2% (+0.6 to +1.8). Cause: Amendment 5: the governor's own core (written before any run on it, 20 October 2026) (`docs/BATCH_QUEUE_PREREGISTRATION.md`). The knob: the compass band's upper cushion on this muscle or the give-back trial's tolerance; where no setting brings the row EVEN in the toy gate, the muscle stays native on this stack.
- **the orchestrator’s own cost.** The Kubernetes scheduler with the operator's bin-packing profile under the batch stream, the four ways (omni-v12, `results/live/V12_BINPACK_4W_VS.md`): time over the response line (% of samples), A B C +21.1% (+5.3 to +36.8) · +37.2% (+12.4 to +61.9) · +29.8% (+1.0 to +58.6). Cause: Binpack 4W Vs (omni-v12, `V12_BINPACK_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_BOTH.md`): pod start wait, total (s), A B C +80.9% (+32.7 to +129.0) · +68.8% (+12.8 to +124.7) · +71.8% (+31.7 to +112.0). Cause: Crane 4W Both (omni-v12, `V12_CRANE_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_ORCH.md`): response time (ms), mean, A B C +2.6% (+0.7 to +4.4) · +3.2% (+0.5 to +5.9) · +2.9% (+0.9 to +5.0). Cause: Crane 4W Orch (omni-v12, `V12_CRANE_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Crane (Effective HPA, on Kubernetes 1.25) under the batch stream, the four ways (omni-v12, `results/live/V12_CRANE_4W_VS.md`): pod start wait, total (s), A B C +60.3% (+28.4 to +92.2) · +56.1% (+0.6 to +111.6) · +51.9% (+20.4 to +83.4). Cause: Crane 4W Vs (omni-v12, `V12_CRANE_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): response time (ms), 95th percentile, A B C +41.9%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(+11.7 to +72.1) · +45.9% (+20.6 to +71.1) · +57.5% (+32.2 to +82.7). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): response time (ms), 99th percentile, A B C +45.2% (+12.1 to +78.2) · +48.9% (+22.4 to +75.5) · +62.1% (+33.1 to +91.1). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): batch: queue finished (s), A B C +5.1% (+1.7 to +8.5) · +6.9% (+3.4 to +10.3) · +5.1% (+3.0 to +7.3). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +1753.8% (+1040.8 to +2466.9) · +1626.7% (+910.1 to +2343.3) · +915.8% (+807.5 to +1024.0). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +550.0% (+38.6 to +1061.4) · +650.0% (+129.0 to +1171.0) · +128.6% (+70.6 to +186.6). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C +48.0% (+17.8 to +78.1) · +51.3% (+28.0 to +74.7) · +52.5% (+31.0 to +73.9). Cause: Kai 4W Both (omni-v12, `V12_KAI_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), mean, A B C +13.2% (+1.6 to +24.8) · +15.8% (+8.1 to +23.5) · +18.3% (+7.5 to +29.1). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), 95th percentile, A B C +38.1%

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(+7.2 to +69.0) · +41.1% (+19.5 to +62.6) · +50.8% (+22.6 to +79.0). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): response time (ms), 99th percentile, A B C +39.9% (+7.8 to +72.1) · +44.0% (+23.2 to +64.9) · +61.3% (+30.8 to +91.7). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): batch: queue finished (s), A B C +4.2% (+1.6 to +6.9) · +5.7% (+2.8 to +8.6) · +4.6% (+2.7 to +6.4). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +1461.5% (+1162.4 to +1760.7) · +1613.3% (+1157.7 to +2069.0) · +968.4% (+828.5 to +1108.4). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +300.0% (+115.3 to +484.7) · +550.0% (+225.7 to +874.3) · +100.0% (+31.0 to +169.0). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** The KAI scheduler under the batch stream, the four ways (omni-v12, `results/live/V12_KAI_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C +40.1% (+16.1 to +64.0) · +42.1% (+24.1 to +60.0) · +45.2% (+26.8 to +63.6). Cause: Kai 4W Orch (omni-v12, `V12_KAI_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), mean, A B C +45.7% (+15.8 to +75.5) · +33.1% (+14.0 to +52.3) · +25.0% (+7.6 to +42.4). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), 95th

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

percentile, A B C +81.3% (+33.4 to +129.1) · +62.1% (+27.0 to +97.3) · +38.8% (+13.8 to +63.9). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): response time (ms), 99th percentile, A B C +73.4% (+34.4 to +112.5) · +64.1% (+30.5 to +97.7) · +36.5% (+13.7 to +59.4). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): time over the response line (% of samples), A B C +17.9% (+8.8 to +27.1) · +12.6% (+0.3 to +25.0) · +22.4% (+7.1 to +37.8). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C +45.1% (+21.3 to +68.9) · +34.5% (+21.0 to +48.0) · +30.8% (+18.3 to +43.2). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): batch: queue finished (s), A B C +16.5% (+7.4 to +25.6) · +15.7% (+8.5 to +23.0) · +14.3% (+5.7 to +22.9). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +180.0% (+82.2 to +277.8) · +183.3% (+52.1 to +314.5) · +24360.0% (+1893.1 to +46826.9). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C +83.0% (+40.0 to +126.1) · +78.2% (+42.6 to +113.8) · +57.9% (+24.1 to +91.8). Cause: Karmada 4W Both (omni-v12, `V12_KARMADA_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), mean, A B C +27.9% (+13.5 to +42.3) · +20.4% (+10.8 to +30.0) · +21.6% (+7.9 to +35.4). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), 95th percentile, A B C +51.1% (+21.9 to +80.2) · +38.9% (+18.4 to +59.5) · +35.4% (+13.9 to +56.9). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): response time (ms), 99th percentile, A B C +54.9% (+23.6 to +86.2) · +47.7% (+25.1 to +70.2) · +31.3% (+13.1 to +49.6). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C +34.3% (+13.3 to +55.3) · +22.1% (+12.1 to +32.1) · +31.7% (+8.1 to +55.3). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): batch: queue finished (s), A B C +11.7% (+5.1 to +18.3) · +11.2% (+6.1 to +16.2) · +11.0% (+5.0 to +16.9). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Karmada on the group of two clusters under the batch stream, the four ways (omni-v12, `results/live/V12_KARMADA_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C +58.2% (+28.9 to +87.5) · +56.1% (+30.6 to +81.6) · +44.2% (+20.7 to +67.7). Cause: Karmada 4W Orch (omni-v12, `V12_KARMADA_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): batch: queue finished (s), A B C +79.6% (+57.4 to +101.8) · +82.5% (+72.2 to +92.7) · +90.2% (+71.9 to +108.5). Cause: Koordinator 4W Both (omni-v12, `V12_KOORDINATOR_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`).

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +332575.0% (+204178.2 to +460971.8) · +701433.3% (+502619.2 to +900247.5) · +227980.0% (+149811.1 to +306148.9). Cause: Koordinator 4W Both (omni-v12, `V12_KOORDINATOR_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +172733.3% (+104746.8 to +240719.9) · +763800.0% (+530043.0 to +997557.0) · +136780.0% (+86044.3 to +187515.7). Cause: Koordinator 4W Both (omni-v12, `V12_KOORDINATOR_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C +1607.8% (+1081.3 to +2134.4) · +849.4% (+675.6 to +1023.2) · +1322.9% (+1027.8 to +1618.1). Cause: Koordinator 4W Both (omni-v12, `V12_KOORDINATOR_4W_BOTH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): batch: queue finished (s), A B C +76.5% (+54.1 to +98.9) · +82.5% (+72.8 to +92.1) · +86.9% (+68.4 to +105.4). Cause: Koordinator 4W Orch (omni-v12, `V12_KOORDINATOR_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +310637.5% (+166814.0 to +454461.0) · +686083.3% (+439006.5 to +933160.2) · +233933.3% (+162122.7 to +305743.9). Cause: Koordinator 4W Orch (omni-v12, `V12_KOORDINATOR_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +171200.0% (+91149.9 to +251250.1) · +797900.0% (+545467.5 to +1050332.5) · +139600.0% (+101748.5 to +177451.5). Cause: Koordinator 4W Orch (omni-v12, `V12_KOORDINATOR_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob:

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C +1512.7% (+1011.0 to +2014.4) · +831.8% (+691.5 to +972.0) · +1276.6% (+979.1 to +1574.2). Cause: Koordinator 4W Orch (omni-v12, `V12_KOORDINATOR_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): response time (ms), 95th percentile, A B C +191.2% (+63.6 to +318.8) · +485.5% (+212.9 to +758.0) · +285.6% (+106.4 to +464.8). Cause: Koordinator 4W Vs (omni-v12, `V12_KOORDINATOR_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): response time (ms), 99th percentile, A B C +181.3% (+56.9 to +305.8) · +481.6% (+212.0 to +751.1) · +285.0% (+116.9 to +453.1). Cause: Koordinator 4W Vs (omni-v12, `V12_KOORDINATOR_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Koordinator under the batch stream, the four ways (omni-v12, `results/live/V12_KOORDINATOR_4W_VS.md`): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C +4.9 (+0.7503 to +9.05) · +8900.0% (+4446.3 to +13353.7) · +9500.0% (+6140.7 to +12859.3). Cause: Koordinator 4W Vs (omni-v12, `V12_KOORDINATOR_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): response time (ms), mean, A B C +85.7% (+68.2 to +103.3) · +85.1% (+68.5 to +101.7) · +67.0% (+49.1 to +84.9). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): time over the response line (% of samples), A B C +212.8% (+175.6 to +250.1) · +223.2% (+172.9 to +273.6) · +207.9% (+172.9 to +243.0). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): alert minutes (whole minutes whose

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

95th percentile, failed requests counted over, was over the line), A B C +202.8% (+146.0 to +259.6) · +216.4% (+168.3 to +264.6) · +157.0% (+115.5 to +198.5). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): batch: queue finished (s), A B C +89.3% (+79.7 to +98.9) · +80.1% (+69.7 to +90.5) · +80.8% (+71.6 to +90.1). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +1967119.6% (+1111934.3 to +2822304.9) · +1287745.8% (+521467.2 to +2054024.4) · +1250833.4% (+747221.5 to +1754445.3). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +1001696.3% (+623421.0 to +1379971.7) · +230987.0% (+116467.0 to +345506.9) · +285052.5% (+193399.9 to +376705.2). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C +1067.6% (+891.6 to +1243.7) · +910.0% (+690.2 to +1129.8) · +774.9% (+649.7 to +900.2). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_BOTH.md`): queue: jobs unfinished at the window's end, A B C +13.44 (+7.163 to +19.73) · +11.3 (+3.858 to +18.74) · +14.5 (+8.326 to +20.67). Cause: Kuberay 4W Both (omni-v12, `V12_KUBERAY_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): response time (ms), mean, A B C +87.9% (+61.0 to +114.8) · +86.6% (+61.9 to +111.4) · +70.6% (+49.8 to +91.5). Cause: Kuberay 4W

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): time over the response line (% of samples), A B C +221.2% (+183.5 to +258.9) · +234.2% (+179.7 to +288.8) · +217.5% (+182.6 to +252.4). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): alert minutes (whole minutes whose 95th percentile, failed requests counted over, was over the line), A B C +204.2% (+145.9 to +262.4) · +219.2% (+169.2 to +269.1) · +159.1% (+116.0 to +202.3). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): batch: queue finished (s), A B C +90.4% (+80.4 to +100.4) · +80.6% (+70.9 to +90.3) · +81.4% (+72.4 to +90.3). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): queue: job wait, submitted to its first pod running, mean (s), A B C +1959438.6% (+1137492.1 to +2781385.0) · +1280430.0% (+511805.8 to +2049054.2) · +1238207.0% (+751232.1 to +1725181.9). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +1015791.5% (+684899.6 to +1346683.4) · +228574.2% (+114225.9 to +342922.5) · +286211.2% (+190609.6 to +381812.7). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, results/live/V12_KUBERAY_4W_ORCH.md): queue: job turnaround, submitted to done, mean (s), A B C +1067.4% (+882.4 to +1252.3) · +927.0% (+727.3 to +1126.7) · +779.3% (+655.6 to +902.9). Cause: Kuberay 4W Orch (omni-v12, V12_KUBERAY_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

own cost.

- **the orchestrator’s own cost.** KubeRay (the Ray autoscaler) under the batch stream, the four ways (omni-v12, `results/live/V12_KUBERAY_4W_ORCH.md`): queue: jobs unfinished at the window's end, A B C +14.33 (+8.22 to +20.45) · +11.1 (+3.755 to +18.44) · +14.6 (+8.613 to +20.59). Cause: Kuberay 4W Orch (omni-v12, `V12_KUBERAY_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): response time (ms), 95th percentile, A B C +9.5% (+1.9 to +17.1) · +13.4% (+3.4 to +23.4) · +15.4% (+7.8 to +22.9). Cause: Kueue 4W Both (omni-v12, `V12_KUEUE_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): batch: queue finished (s), A B C +1.0% (+0.3 to +1.7) · +1.6% (+0.8 to +2.5) · +1.9% (+1.1 to +2.8). Cause: Kueue 4W Both (omni-v12, `V12_KUEUE_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_BOTH.md`): queue: job turnaround, submitted to done, mean (s), A B C +12.5% (+3.2 to +21.8) · +13.8% (+6.4 to +21.3) · +16.6% (+8.6 to +24.5). Cause: Kueue 4W Both (omni-v12, `V12_KUEUE_4W_BOTH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): response time (ms), mean, A B C +7.3% (+0.0 to +14.5) · +6.4% (+1.8 to +11.0) · +8.9% (+3.7 to +14.1). Cause: Kueue 4W Orch (omni-v12, `V12_KUEUE_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): response time (ms), 95th percentile, A B C +12.8% (+0.3 to +25.2) · +10.0% (+1.4 to +18.7) · +10.2% (+2.9 to +17.5). Cause: Kueue 4W Orch (omni-v12, `V12_KUEUE_4W_ORCH.md`, 9 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, `results/live/V12_KUEUE_4W_ORCH.md`): queue: job wait, submitted to its first pod running, mean (s), A B C +178.6% (+75.7 to +281.5) · +118.2% (+22.2 to +214.1) · +537.5% (+313.8 to +761.2). Cause: Kueue 4W Orch (omni-v12, `V12_KUEUE_4W_ORCH.md`, 9 paired repetitions a run)

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Kueue under the batch stream, the four ways (omni-v12, results/live/V12_KUEUE_4W_ORCH.md): queue: job turnaround, submitted to done, mean (s), A B C +11.5% (+0.3 to +22.7) · +9.3% (+2.0 to +16.5) · +12.7% (+5.4 to +19.9). Cause: Kueue 4W Orch (omni-v12, V12_KUEUE_4W_ORCH.md, 9 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_BOTH.md): batch: queue finished (s), A B C +1.8% (+0.8 to +2.8) · +1.7% (+0.4 to +2.9) · +1.8% (+0.4 to +3.2). Cause: Volcano 4W Both (omni-v12, V12_VOLCANO_4W_BOTH.md, 10 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_BOTH.md): queue: job wait, submitted to its first pod running, mean (s), A B C +1126.7% (+1020.2 to +1233.2) · +1088.9% (+690.1 to +1487.7) · +1416.7% (+986.8 to +1846.5). Cause: Volcano 4W Both (omni-v12, V12_VOLCANO_4W_BOTH.md, 10 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_BOTH.md): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +333.3% (+174.4 to +492.3) · +450.0% (+38.7 to +861.3) · +550.0% (+436.9 to +663.1). Cause: Volcano 4W Both (omni-v12, V12_VOLCANO_4W_BOTH.md, 10 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_BOTH.md): queue: job turnaround, submitted to done, mean (s), A B C +32.1% (+13.4 to +50.8) · +19.7% (+5.5 to +33.9) · +26.0% (+8.1 to +43.9). Cause: Volcano 4W Both (omni-v12, V12_VOLCANO_4W_BOTH.md, 10 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_ORCH.md): batch: queue finished (s), A B C +1.2% (+0.6 to +1.8) · +1.4% (+0.4 to +2.5) · +1.4% (+0.3 to +2.6). Cause: Volcano 4W Orch (omni-v12, V12_VOLCANO_4W_ORCH.md, 10 paired repetitions a run) (docs/POWER.md). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, results/live/V12_VOLCANO_4W_ORCH.md): queue: job wait, submitted to its first pod running,

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

mean (s), A B C +1053.3% (+883.9 to +1222.8) · +866.7% (+664.7 to +1068.6) · +1491.7% (+1085.0 to +1898.3). Cause: Volcano 4W Orch (omni-v12, `V12_VOLCANO_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): queue: job wait, submitted to its first pod running, 95th percentile (s), A B C +366.7% (+231.3 to +502.0) · +275.0% (+78.2 to +471.8) · +500.0% (+331.4 to +668.6). Cause: Volcano 4W Orch (omni-v12, `V12_VOLCANO_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_ORCH.md`): queue: job turnaround, submitted to done, mean (s), A B C +24.2% (+10.9 to +37.5) · +17.6% (+5.6 to +29.5) · +23.2% (+6.9 to +39.4). Cause: Volcano 4W Orch (omni-v12, `V12_VOLCANO_4W_ORCH.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): response time (ms), 95th percentile, A B C +92.6% (+46.7 to +138.5) · +31.9% (+10.8 to +53.0) · +50.9% (+23.2 to +78.6). Cause: Volcano 4W Vs (omni-v12, `V12_VOLCANO_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.
- **the orchestrator’s own cost.** Volcano under the batch stream, the four ways (omni-v12, `results/live/V12_VOLCANO_4W_VS.md`): time over the response line (% of samples), A B C +124.1% (+51.4 to +196.7) · +50.2% (+5.7 to +94.8) · +54.8% (+19.1 to +90.4). Cause: Volcano 4W Vs (omni-v12, `V12_VOLCANO_4W_VS.md`, 10 paired repetitions a run) (`docs/POWER.md`). The knob: nothing of Omni-Compass to turn: the orchestrator's own cost.

Appendix D. Glossary

- **Native**: the system under its own controllers alone. **Omni-Compass** (the omni arm): the same system with Omni-Compass on top of those controllers.
- **Muscle**: one knob of the system a native controller turns (replicas, machines, a power limit, a frequency ceiling, a cache size). **Organism**: a body of muscles governed as one.
- **Compass law**: the band with 5% cushions and two antagonist forces, smooth (tanh), that moves a muscle's setting (`omnicompass/compass_law.py`).
- **Brain**: the engine's six state variables, integrated by the fourth-order Runge–Kutta step every decision (`omnicompass/core.py`). **Verdict**: the give-back trial judged against native

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com

(`omnicompass/verdict.py`).

- **Kill switch**: the master switch's instant hand-back of every setting to native, for security. **Lease**: the gradual walk back to native after a crash.
- **The max / the tune-down**: the dial's two settings. **Power given**: a resource spent that rose, shown beside what it bought, never a LOSS.
- **A, B and C**: a benchmark's three runs; a reading counts when it holds the same way in all three.
- **Evidence classes**: P a physical meter, L live software, S our own models.

Contact

The Omni-Compass LLC, main office infrastructure. Toll-free direct: 1-888-339-9551. System route asset: 1-888-DEX-YLL-1. Landline, registered in Florida: 321-746-4411. www.omni-compass.com

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with `LICENSE`, `NOTICE` *and* `DISCLOSURES.md`. *All patents, copyrights and trademarks filed in the USA. www.omni-compass.com*

© 2026 The Omni-Compass LLC. All rights reserved. Evaluation and Simulation use only. Any commercial use, commercialization, monetization, production use, redistribution or hosted service of any part of Omni-Compass requires a signed, paid Omni-Compass Enterprise License from The Omni-Compass LLC. Every copy, export, report and printout carries this notice with LICENSE, NOTICE and DISCLOSURES.md. All patents, copyrights and trademarks filed in the USA. www.omni-compass.com